

Universite / Ecole d'Ingenieurs

Filiere BDIA1 — Big Data & Intelligence Artificielle



---

# Systeme Intelligent de Controle Qualite des Impellers

---

*Detection automatique des defauts de fonderie  
par Apprentissage Profond et Transfer Learning*

**Rapport de projet de fin de module**  
***Module : Apprentissage Profond et Vision par Ordinateur***

**Realise par :**

Ayoub EL-MOUJAHID

Mohamed ERRAHMOUNI

Soulama Doubie Judicael Noe

Hazem DOUZI

**Encadre par :**

Prof. Dr Naoual ATTAOUI

Annee universitaire **2025–2026**

# Remerciements

Au terme de ce projet de fin de module, nous tenons à exprimer notre profonde gratitude à toutes les personnes qui, par leur soutien, leurs conseils et leur disponibilité, ont contribué de près ou de loin à son aboutissement.

Nos remerciements les plus sincères s'adressent en premier lieu à notre encadrante, **Madame le Professeur Dr Naoual Attaoui**, pour la qualité de son encadrement, la rigueur scientifique de ses orientations et la confiance qu'elle nous a accordée tout au long de ce travail. Sa pédagogie et ses remarques éclairées ont constitué un guide précieux durant chaque étape de la réalisation.

Nous remercions également l'ensemble du corps professoral de la filière **BDIA1 (Big Data & Intelligence Artificielle)** pour la qualité des enseignements dispensés, qui ont jeté les fondations théoriques et pratiques sur lesquelles repose ce projet.

Notre reconnaissance va aussi à la communauté open-source — en particulier aux équipes de **TENSORFLOW**, **KERAS**, **PYTORCH**, **SCIKIT-LEARN** et **MATPLOTLIB** — ainsi qu'à Ravirajsinh Dabhi qui a publié sur Kaggle le jeu de données *Casting Defect Detection* sans lequel ce travail n'aurait pas été possible.

Enfin, nous adressons un remerciement chaleureux à nos familles et à nos camarades de promotion pour leur soutien moral indéfectible et l'enrichissement intellectuel mutuel qui a caractérisé notre année universitaire.

*Les auteurs*

# Resume

Dans le secteur de la fonderie, le contrôle qualité des pièces métalliques — en particulier des *impellers* (roues de pompes centrifuges) — représente un enjeu industriel majeur. Réalisé traditionnellement de manière visuelle par des opérateurs humains, ce contrôle souffre d’une triple limite : il est lent, subjectif et sujet à la fatigue. Une seule pièce defectueuse non détectée peut provoquer une défaillance en service avec des conséquences coûteuses pour le client final.

Le présent rapport propose un **système intelligent de contrôle qualité automatique** fondé sur la *vision par ordinateur* et l’*apprentissage profond*. Trois familles de modèles sont développées, comparées et discutées suivant une démarche progressive : (i) un **Perceptron Multi-Couches (MLP)** servant de *baseline*, (ii) un **Reseau de Neurones Convolutif (CNN)** entraîné de zéro, et (iii) un modèle de **Transfer Learning** fondé sur l’architecture EFFICIENTNETB0 pré-entraînée sur IMAGENET.

Une attention particulière est portée à la chaîne de prétraitement, développée en double implémentation TENSORFLOW (`tf.data`) et PYTORCH (`DataLoader`) afin de garantir reproductibilité, modularité et portabilité. Cette chaîne intègre la normalisation *ImageNet*, des transformations *color balancing* (white balance, égalisation d’histogramme, channel shuffle), des augmentations géométriques et photométriques, ainsi qu’une gestion explicite du déséquilibre des classes par *class weights* ou `WeightedRandomSampler`.

Le modèle final atteint une **accuracy de 99,44%** et un **AUC de 0,9993** sur l’ensemble de test, avec un *recall* de **99,34%** sur la classe defectueuse — métrique critique en contrôle qualité industriel. Ces performances, obtenues avec seulement 6 600 images d’entraînement, démontrent la pertinence du *Transfer Learning* pour les problèmes de classification industrielle à faible volume de données.

**Mots-cles :** Contrôle qualité, Fonderie, Impeller, Apprentissage profond, Réseaux de neurones convolutifs, Transfer Learning, EfficientNet, TensorFlow, PyTorch, Vision par ordinateur, Détection de défauts.



# Abstract

In the foundry industry, quality control of metallic parts — particularly *impellers* (centrifugal pump wheels) — represents a major industrial challenge. Traditionally performed visually by human operators, such inspection suffers from three intrinsic limitations : it is slow, subjective, and prone to fatigue-induced errors. A single undetected defective part can lead to in-service failure with costly consequences for the end customer.

This report proposes an **intelligent automatic quality control system** based on *computer vision* and *deep learning*. Three model families are developed, compared, and discussed following a progressive approach : (i) a **Multi-Layer Perceptron (MLP)** as a baseline, (ii) a **Convolutional Neural Network (CNN)** trained from scratch, and (iii) a **Transfer Learning** model based on the EFFICIENTNETB0 architecture pre-trained on IMAGENET.

Special attention is devoted to the preprocessing pipeline, developed in dual implementation TENSORFLOW (`tf.data`) and PYTORCH (`DataLoader`) to ensure reproducibility, modularity and portability. The pipeline integrates *ImageNet* normalization, *color balancing* transformations (white balance, histogram equalization, channel shuffle), geometric and photometric augmentations, as well as explicit class imbalance handling through *class weights* or `WeightedRandomSampler`.

The final model achieves **99.44% accuracy** and an **AUC of 0.9993** on the test set, with **99.34% recall** on the defective class — the critical metric in industrial quality control. These results, obtained with only 6,600 training images, demonstrate the relevance of *Transfer Learning* for industrial classification problems with limited data.

**Keywords :** Quality Control, Foundry, Impeller, Deep Learning, Convolutional Neural Networks, Transfer Learning, EfficientNet, TensorFlow, PyTorch, Computer Vision, Defect Detection.

# Table des matières

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| Remerciements                                                         | 1         |
| Resume                                                                | 2         |
| Abstract                                                              | 3         |
| Liste des figures                                                     | 8         |
| Liste des tableaux                                                    | 9         |
| Liste des acronymes                                                   | 10        |
| <b>1 Introduction generale</b>                                        | <b>11</b> |
| 1.1 Contexte general . . . . .                                        | 11        |
| 1.2 Problematique . . . . .                                           | 11        |
| 1.3 Objectifs du projet . . . . .                                     | 12        |
| 1.4 Methodologie generale . . . . .                                   | 12        |
| 1.5 Organisation du rapport . . . . .                                 | 13        |
| <b>2 Etat de l'art</b>                                                | <b>14</b> |
| 2.1 Controle qualite industriel par vision artificielle . . . . .     | 14        |
| 2.2 Architectures convolutives marquantes . . . . .                   | 14        |
| 2.2.1 Les jalons historiques . . . . .                                | 14        |
| 2.2.2 EfficientNet : etat de l'art recent . . . . .                   | 15        |
| 2.3 Transfer Learning . . . . .                                       | 15        |
| 2.4 Travaux relatifs a la detection de defauts en fonderie . . . . .  | 16        |
| <b>3 Procede de fabrication et typologie des defauts</b>              | <b>17</b> |
| 3.1 Procede de fonderie en moule carapace . . . . .                   | 17        |
| 3.1.1 Etape 1 : Conception (Design of pump) . . . . .                 | 17        |
| 3.1.2 Etape 2 : Fabrication du moule carapace (Shell mould) . . . . . | 17        |
| 3.1.3 Etape 3 : Coulee (Shell mould casting process) . . . . .        | 17        |
| 3.2 Typologie des defauts observes . . . . .                          | 18        |
| 3.3 Objectif specifique du projet . . . . .                           | 19        |
| <b>4 Acquisition et organisation des donnees</b>                      | <b>20</b> |
| 4.1 Choix du jeu de donnees . . . . .                                 | 20        |

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| 4.2      | Description du dataset . . . . .                                  | 20        |
| 4.3      | Repartition train / validation / test . . . . .                   | 21        |
| 4.4      | Volumes et desequilibre des classes . . . . .                     | 22        |
| <b>5</b> | <b>Pretraitement des images</b>                                   | <b>23</b> |
| 5.1      | Philosophie generale du pipeline . . . . .                        | 23        |
| 5.2      | Vue d'ensemble du pipeline TensorFlow (10 sections) . . . . .     | 23        |
| 5.2.1    | Reproductibilite et detection du <i>device</i> . . . . .          | 24        |
| 5.2.2    | Color balancing : compatibilite <i>graph</i> TensorFlow . . . . . | 25        |
| 5.2.3    | Normalisation ImageNet . . . . .                                  | 26        |
| 5.2.4    | Comparaison train vs eval . . . . .                               | 26        |
| 5.2.5    | Chargement et ponderation des classes . . . . .                   | 27        |
| 5.2.6    | Composition finale des datasets . . . . .                         | 28        |
| 5.2.7    | Configuration centralisee et utilitaires . . . . .                | 28        |
| 5.3      | Pipeline equivalent en PyTorch . . . . .                          | 29        |
| 5.3.1    | Reproductibilite et device . . . . .                              | 29        |
| 5.3.2    | Color balancing personnalise . . . . .                            | 30        |
| 5.3.3    | Pipeline complet d'augmentations . . . . .                        | 30        |
| 5.3.4    | Gestion du desequilibre des classes . . . . .                     | 31        |
| 5.3.5    | DataLoaders et utilitaires . . . . .                              | 31        |
| <b>6</b> | <b>Modelisation et experimentations</b>                           | <b>33</b> |
| 6.1      | Modele 1 : Perceptron Multi-Couches (MLP) . . . . .               | 33        |
| 6.1.1    | Motivation . . . . .                                              | 33        |
| 6.1.2    | Representation des donnees : le Flatten . . . . .                 | 33        |
| 6.1.3    | Architecture du MLP . . . . .                                     | 34        |
| 6.1.4    | Mecanismes de regularisation . . . . .                            | 35        |
| 6.1.5    | Resultats du MLP . . . . .                                        | 35        |
| 6.1.6    | Analyse critique du MLP . . . . .                                 | 36        |
| 6.2      | Modele 2 : CNN entraine de zero . . . . .                         | 36        |
| 6.2.1    | Principe et fondements theoriques . . . . .                       | 37        |
| 6.2.2    | Architecture du CNN propose . . . . .                             | 37        |
| 6.2.3    | Compilation et regularisation . . . . .                           | 38        |
| 6.2.4    | Entrainement . . . . .                                            | 38        |
| 6.2.5    | Evaluation et diagnostic d'overfitting . . . . .                  | 39        |
| 6.2.6    | Regularisation par augmentation des donnees . . . . .             | 40        |
| 6.2.7    | Conclusion intermediaire sur le CNN . . . . .                     | 41        |
| 6.3      | Modele 3 : Transfer Learning avec EfficientNetB0 . . . . .        | 41        |
| 6.3.1    | Principe du Transfer Learning . . . . .                           | 41        |
| 6.3.2    | Strategie en deux phases . . . . .                                | 42        |

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| 6.3.3    | Architecture choisie : EfficientNetB0 . . . . .    | 43        |
| 6.3.4    | Source des données et préparation . . . . .        | 44        |
| 6.3.5    | Hyperparamètres . . . . .                          | 46        |
| 6.3.6    | Construction du modèle . . . . .                   | 46        |
| 6.3.7    | Phase 1 : Extraction de caractéristiques . . . . . | 47        |
| 6.3.8    | Phase 2 : Fine-tuning . . . . .                    | 47        |
| 6.3.9    | Courbes d'entraînement consolidées . . . . .       | 48        |
| <b>7</b> | <b>Evaluation finale et discussion</b>             | <b>50</b> |
| 7.1      | Protocole d'évaluation . . . . .                   | 50        |
| 7.2      | Métriques industrielles clés . . . . .             | 50        |
| 7.2.1    | Rappel des définitions . . . . .                   | 50        |
| 7.2.2    | Résultats sur le test set . . . . .                | 51        |
| 7.2.3    | Asymétrie des coûts d'erreur . . . . .             | 51        |
| 7.3      | Matrice de confusion . . . . .                     | 52        |
| 7.4      | Courbes ROC et Precision-Recall . . . . .          | 52        |
| 7.5      | Analyse fine des erreurs . . . . .                 | 53        |
| 7.6      | Comparaison globale des trois modèles . . . . .    | 54        |
| 7.7      | Discussion industrielle . . . . .                  | 55        |
| 7.7.1    | Adequation au déploiement . . . . .                | 55        |
| 7.7.2    | Limites résiduelles . . . . .                      | 55        |
| <b>8</b> | <b>Conclusion générale et perspectives</b>         | <b>57</b> |
| 8.1      | Bilan du projet . . . . .                          | 57        |
| 8.2      | Apports personnels . . . . .                       | 58        |
| 8.3      | Perspectives . . . . .                             | 58        |
| 8.3.1    | Extension fonctionnelle . . . . .                  | 58        |
| 8.3.2    | Industrialisation . . . . .                        | 59        |
| 8.3.3    | Amélioration continue . . . . .                    | 59        |
| 8.3.4    | Ouverture vers l'Industrie 4.0 . . . . .           | 59        |
|          | <b>Bibliographie</b>                               | <b>62</b> |

# Table des figures

|      |                                                                                                                                          |    |
|------|------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1  | Etapes du procede de fonderie en moule carapace. . . . .                                                                                 | 18 |
| 3.2  | Exemples typiques de defauts compares a une piece conforme. . . . .                                                                      | 19 |
| 4.1  | Apercu visuel du dataset <i>Casting Defect Detection</i> (Kaggle). . . . .                                                               | 21 |
| 5.1  | Bandeau de presentation du pipeline de pretraitement TensorFlow. . . . .                                                                 | 24 |
| 5.2  | Sorties des utilitaires de controle du pipeline. . . . .                                                                                 | 29 |
| 6.1  | Aplatissement ( <i>Flatten</i> ) d'une image en vecteur 1D. . . . .                                                                      | 33 |
| 6.2  | Schema global de l'architecture MLP. . . . .                                                                                             | 34 |
| 6.3  | Performance du MLP sur le jeu de test. . . . .                                                                                           | 35 |
| 6.4  | Architecture du CNN entraine de zero. . . . .                                                                                            | 37 |
| 6.5  | Compilation et configuration du CNN avec callbacks de regularisation. . .                                                                | 38 |
| 6.6  | Entrainement du CNN sur 30 epoques. . . . .                                                                                              | 39 |
| 6.7  | Courbes d'entrainement du CNN avant data augmentation : signe clair de<br>sur-apprentissage. . . . .                                     | 39 |
| 6.8  | Effet de la <i>data augmentation</i> sur l'apprentissage. . . . .                                                                        | 40 |
| 6.9  | Synthese visuelle des resultats du CNN. . . . .                                                                                          | 41 |
| 6.10 | Principe general du Transfer Learning : reutilisation des features d'un<br>modele pre-entraine. . . . .                                  | 42 |
| 6.11 | Reutilisation du <i>backbone</i> pre-entraine : la tete originale (1000 classes<br>ImageNet) est remplacee par une tete binaire. . . . . | 43 |
| 6.12 | Architecture detaillee d'EfficientNetB0 (blocs MBConv). . . . .                                                                          | 44 |
| 6.13 | Dataset <i>Casting Defect Detection</i> dans sa version augmentee fournie sur<br>Kaggle. . . . .                                         | 44 |
| 6.14 | Code de chargement et de pretraitement adapte a EfficientNet. . . . .                                                                    | 45 |
| 6.15 | Hyperparametres cles utilises pour les deux phases. . . . .                                                                              | 46 |
| 6.16 | Construction du modele de Transfer Learning. . . . .                                                                                     | 46 |
| 6.17 | Logs d'entrainement Phase 1 - Extraction de caracteristiques. . . . .                                                                    | 47 |
| 6.18 | Logs d'entrainement Phase 2 - Fine-tuning. . . . .                                                                                       | 48 |
| 6.19 | Courbes <i>accuracy</i> , <i>loss</i> et AUC durant les phases 1 et 2. . . . .                                                           | 48 |
| 7.1  | Performance detaillee sur l'ensemble de test (715 images). . . . .                                                                       | 51 |
| 7.2  | Matrice de confusion sur le <i>test set</i> . . . . .                                                                                    | 52 |
| 7.3  | Courbes ROC (gauche) et Precision-Recall (droite) sur le test set. . . . .                                                               | 52 |
| 7.4  | Visualisation des images mal classees : FN a gauche, FP a droite. . . . .                                                                | 54 |

|     |                                                                                                                                               |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------|----|
| 7.5 | Comparaison des courbes ROC — le Transfer Learning (rouge) discrimine<br>quasi-parfaitement les deux classes, quel que soit le seuil. . . . . | 55 |
| 8.1 | Synthese visuelle du resultat final. . . . .                                                                                                  | 57 |

# Liste des tableaux

|     |                                                                                       |    |
|-----|---------------------------------------------------------------------------------------|----|
| 4.1 | Repartition train/validation/test du dataset. . . . .                                 | 21 |
| 5.1 | Constantes globales partagees par tous les notebooks. . . . .                         | 25 |
| 5.2 | Comparaison des transformations <i>train</i> vs <i>eval</i> . . . . .                 | 27 |
| 5.3 | Configuration des trois <b>DataLoaders</b> PyTorch. . . . .                           | 31 |
| 6.1 | Resultats du MLP sur l'ensemble de test. . . . .                                      | 36 |
| 6.2 | Strategie de Transfer Learning a deux etages. . . . .                                 | 42 |
| 7.1 | Comparatif des trois approches. <sup>†</sup> apres <i>data augmentation</i> . . . . . | 54 |

# Liste des acronymes

**AUC** Area Under the Curve.

**CAO** Conception Assistee par Ordinateur.

**CNN** Convolutional Neural Network.

**FN** Faux Negatif.

**FP** Faux Positif.

**GPU** Graphics Processing Unit.

**MLP** Multi-Layer Perceptron.

**ReLU** Rectified Linear Unit.

**TFLite** TensorFlow Lite.

**TL** Transfer Learning (apprentissage par transfert).



# 1. Introduction generale

## 1.1 Contexte general

La quatrieme revolution industrielle, communement designee sous le terme *Industrie 4.0*, redefinit en profondeur les processus de production manufacturiere. Au cœur de cette transformation se trouve l'integration massive de technologies numeriques — objets connectes, jumeaux numeriques, robotique collaborative et, plus particulierement, *intelligence artificielle* — au sein des chaines de fabrication. Dans ce contexte, le **contrôle qualité automatique** apparait comme l'une des applications a plus forte valeur ajoutee : il conditionne directement la fiabilite des produits livres, la satisfaction client et la rentabilite des installations.

La fonderie, secteur historique et strategique de l'industrie metallurgique, n'echappe pas a cette mutation. Les pieces issues du moulage — particulierement les composants critiques tels que les *impellers* de pompes centrifuges — doivent satisfaire a des specifications dimensionnelles et de surface tres exigeantes. Tout ecart (fissure, deformation, bavure, porosite) peut compromettre la fonction hydraulique de la pompe et entrainer une defaillance prematuree, parfois catastrophique, en service.

## 1.2 Problematique

Le controle qualite des impellers en sortie de chaine de fonderie est aujourd'hui realise selon deux approches :

- **Inspection visuelle manuelle.** Un operateur examine chaque piece a l'œil nu et la classe comme conforme ou defectueuse. Cette methode est universelle mais souffre de trois limites majeures : *lenteur* (cadence de production limitee), *subjectivite* (variabilite inter-operateurs et intra-operateur), et *fatigue* (taux d'erreur croissant en fin de poste).
- **Contrôle non destructif (CND) instrumente.** Radiographie, ressuage, ultrasons. Tres precis mais couteux, lents et inadaptés a un controle de masse a 100 %.

Aucune de ces deux approches ne permet a la fois un controle *exhaustif*, *rapide* et *reproductible* en ligne. Cette limitation conduit, dans la pratique industrielle, a l'application

d'un contrôle par échantillonnage statistique — ce qui laisse mécaniquement passer une fraction de pièces defectueuses chez le client.

#### Question de recherche

Est-il possible de concevoir un système automatique, capable de classifier en quelques millisecondes une image de pièce comme **conforme** ou **defectueuse**, avec une précision suffisante pour un déploiement industriel et un *recall* élevé sur la classe defectueuse afin de minimiser les faux négatifs ?

## 1.3 Objectifs du projet

Pour répondre à cette problématique, le présent projet poursuit les objectifs suivants :

1. **Etudier** les principes et l'état de l'art de la classification d'images par apprentissage profond, en particulier les architectures convolutives et le *Transfer Learning*.
2. **Concevoir** une chaîne de prétraitement reproductible et portable, adaptée aux contraintes des images industrielles (variations d'éclairage, angles, textures de surface).
3. **Implémenter et comparer** trois modèles de classification représentatifs d'une progression méthodologique : Multi-Layer Perceptron (MLP), Convolutional Neural Network (CNN) et Transfer Learning sur EFFICIENTNETB0.
4. **Évaluer rigoureusement** les performances obtenues à l'aide de métriques industriellement pertinentes (*accuracy*, *precision*, *recall*, F1-score, Area Under the Curve (AUC)) et d'une analyse fine des erreurs (faux négatifs, faux positifs).
5. **Discuter** les limites du système proposé et formuler des perspectives pour son industrialisation effective (déploiement embarqué, multi-classes, localisation).

## 1.4 Méthodologie générale

La démarche méthodologique adoptée s'inscrit dans une logique *incrémentale* et *comparative* :

1. **Acquisition** d'un jeu de données public reconnu (*Casting Defect Detection*, Kaggle).
2. **Exploration et prétraitement** avec une double implémentation TensorFlow / PyTorch pour valider la robustesse du pipeline.

3. **Modelisation progressive** : chaque modele est entraine, evalue, ses limites identifiees, et la generation suivante est concue pour les depasser.
4. **Evaluation finale** sur un ensemble de test *strictement disjoint*, jamais vu durant l'apprentissage.
5. **Analyse critique** des resultats et des erreurs residuelles dans une perspective de deploiement industriel.

## 1.5 Organisation du rapport

Ce rapport est structure en **huit chapitres** :

### Chapitre 1

La presente introduction generale.

### Chapitre 2

Etat de l'art du controle qualite par vision artificielle et apprentissage profond.

### Chapitre 3

Procede de fonderie en moule carapace et typologie des defauts observes sur les impellers.

### Chapitre 4

Acquisition, organisation et exploration du jeu de donnees.

### Chapitre 5

Conception detaillee du pipeline de pretraitement (TensorFlow et PyTorch).

### Chapitre 6

Modelisation : MLP, CNN et Transfer Learning avec EFFICIENTNETB0.

### Chapitre 7

Evaluation finale, analyse des erreurs et discussion industrielle.

### Chapitre 8

Conclusion generale et perspectives.

## 2. Etat de l'art

### 2.1 Controle qualite industriel par vision artificielle

Le controle qualite par vision artificielle (*Machine Vision*) est une discipline ancienne, dont les premiers systemes industriels remontent aux annees 1980. Historiquement, ces systemes reposaient sur des *regles expertes* fondees sur le traitement d'image classique : seuillage adaptatif, detection de contours (filtres de Sobel, Canny), morphologie mathematique, et descripteurs de texture (matrices de co-occurrence, ondelettes de Gabor). Ces approches, performantes sur des defauts simples et bien definis, montrent rapidement leurs limites des que l'on s'attaque a des defauts *visuellement variables* — tels que les fissures fines, les deformations subtiles ou les bavures heterogenes que l'on rencontre en fonderie.

L'avenement de l'apprentissage profond, et plus precisement des reseaux convolutifs [1], a profondement bouleverse ce paysage. La capacite des CNN a apprendre automatiquement des representations hierarchiques des images — des contours de bas niveau aux concepts semantiques de haut niveau — a permis des progres considerables, comme illustre par la victoire d'ALEXNET [2] a la competition ILSVRC 2012 sur IMAGENET. Depuis, le *deep learning* s'est impose comme l'approche dominante pour la classification d'images, et son adoption dans le controle qualite industriel s'est rapidement generalisee.

### 2.2 Architectures convolutives marquantes

#### 2.2.1 Les jalons historiques

- **LeNet-5** [1] : premier CNN moderne, applique a la reconnaissance de chiffres manuscrits. Pose les bases (convolution, pooling, couches denses).
- **AlexNet** [2] : 8 couches, exploitation du GPU, fonctions d'activation Rectified Linear Unit (ReLU), *dropout*. Marque le debut de l'ere *deep*.
- **VGGNet** [3] : empilement profond (16 a 19 couches) de petits filtres  $3 \times 3$ . Demonstration que la profondeur ameliore la performance.
- **ResNet** [5] : introduction des connexions residuelles (*skip connections*), permettant d'entrainer des reseaux jusqu'a 152 couches sans degradation.
- **Inception / GoogLeNet** [4] : *Inception modules* parallelisant plusieurs tailles de filtres dans un meme bloc.

### 2.2.2 EfficientNet : etat de l'art recent

EFFICIENTNET [6], introduit par Tan et Le en 2019, propose une approche systematique de *compound scaling* : au lieu de faire croitre arbitrairement la profondeur, la largeur ou la resolution d'entree, les auteurs montrent qu'il est optimal de *conjointement* augmenter ces trois dimensions selon un facteur unique  $\phi$  :

$$\text{depth : } d = \alpha^\phi, \quad \text{width : } w = \beta^\phi, \quad \text{resolution : } r = \gamma^\phi \quad (2.1)$$

sous la contrainte  $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ . Le bloc fondamental d'EfficientNet est le *Mobile Inverted Bottleneck* (MBCONV), enrichi de *squeeze-and-excitation* [7]. Le modele EFFICIENTNETB0 compte environ 5,3 millions de parametres et atteint 77,3% de top-1 accuracy sur ImageNet — un excellent compromis performance/cout, particulierement adapte au *Transfer Learning*.

## 2.3 Transfer Learning

L'idee fondamentale du *Transfer Learning* [11, 12] consiste a reutiliser les connaissances apprises par un modele sur une tache source (ici la classification d'IMAGENET, qui contient 1,28 million d'images reparties en 1 000 classes) pour resoudre une tache cible differente mais reliee — dans notre cas, la classification binaire de pieces de fonderie. La justification theorique repose sur la nature *hierarchique* des representations apprises par les CNN : les premieres couches detectent des concepts visuels generiques (contours, textures, gradients de couleur) qui sont *transferables* entre la quasi-totalite des taches d'images naturelles, tandis que les dernieres couches encodent des concepts plus specifiques a la tache source.

En pratique, le Transfer Learning s'opere selon deux strategies, souvent combinees :

1. **Feature extraction** : le *backbone* pre-entraîne est gele, et seule une nouvelle tete de classification est entraînée sur la tache cible. Strategie privilegiee lorsque le dataset cible est petit (< 10 000 images).
2. **Fine-tuning** : une partie ou la totalite du backbone est degelee et reentraîne avec un *learning rate* tres faible afin d'ajuster les representations pre-apprises sans les detruire.

## 2.4 Travaux relatifs a la detection de defauts en fonderie

Plusieurs etudes recentes ont applique le *deep learning* a la detection de defauts dans des contextes industriels analogues : aciers laminés [14], soudures [15], surfaces metalliques generales [16]. Ces travaux convergent sur trois enseignements :

- le Transfer Learning supplante systematiquement l’entrainement de zero pour les datasets de taille moderee ;
- les augmentations photometriques (variations d’eclairage, contraste, saturation) sont essentielles pour la generalisation a des conditions d’usine variables ;
- le *recall* sur la classe defectueuse est la metrique critique : un faux negatif a un cout asymetriquement plus eleve qu’un faux positif.

## 3. Procédé de fabrication et typologie des défauts

### 3.1 Procédé de fonderie en moule carapace

L'*impeller* (turbine ou roue de pompe) est une pièce métallique critique qui transmet l'énergie mécanique au fluide pompe. Son procédé de fabrication par fonderie en moule carapace (*shell mould casting*) se décompose en trois grandes étapes successives.

#### 3.1.1 Etape 1 : Conception (Design of pump)

La pièce est d'abord modélisée en Conception Assistée par Ordinateur (CAO) : une roue à aubes destinée à équiper une pompe centrifuge. Cette étape détermine la géométrie finale, les tolérances dimensionnelles et la spécification de l'état de surface attendu en sortie de fonderie.

#### 3.1.2 Etape 2 : Fabrication du moule carapace (Shell mould)

Un moule en sable aggloméré par résine thermodurcissable est confectionné autour d'un modèle en métal chauffé à environ 250°C. Le sable résine durcit au contact de la chaleur en formant une *carapace* de quelques millimètres d'épaisseur, qui sera ensuite démoulée et utilisée comme empreinte négative de la pièce à couler.

#### 3.1.3 Etape 3 : Coulee (Shell mould casting process)

Plusieurs moules carapaces sont assemblés et serrés dans un cadre métallique. Le métal en fusion (typiquement un alliage d'aluminium, de fonte ou d'acier selon l'application) est ensuite coulé par gravité dans les empreintes, refroidit et se solidifie. Après démoulage, la pièce métallique est nettoyée, ébavurée, puis usinée si nécessaire pour atteindre les tolérances finales. La figure 3.1 illustre les principales étapes industrielles de ce procédé.



(a) Moule carapace assemble et serre dans son cadre, prêt à recevoir le metal en fusion.

(b) Phase de coulee : le metal liquide est verse dans les moules avant solidification.

**Figure 3.1** – Etapes du procede de fonderie en moule carapace.

**Lecture de la figure 3.1.** La sous-figure (a) montre l'arrangement des moules carapaces serres ensemble par un cadre extérieur : cette configuration permet de couler plusieurs pieces simultanement et d'optimiser la productivite. La sous-figure (b) illustre la phase critique de la coulee, ou la qualite finale de la piece se joue : vitesse de coulee, temperature du metal, vitesse de refroidissement et qualite du moule sont autant de parametres susceptibles d'introduire des defauts internes ou de surface.

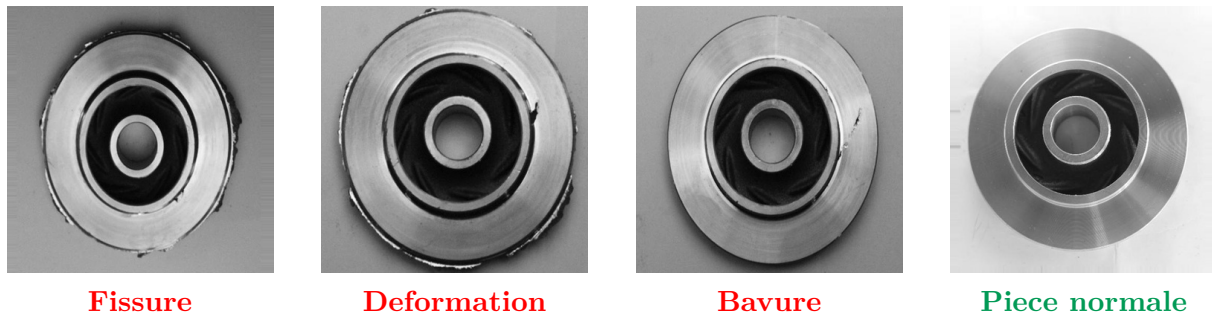
## 3.2 Typologie des defauts observes

Apres refroidissement et demoulage, certaines pieces peuvent presenter des anomalies qui les rendent impropres a l'usage. Les defauts les plus frequemment observes sur les impellers de fonderie sont les suivants :

- **Fissures (cracks).** Discontinuites lineaires, souvent fines, resultant de tensions internes lors du refroidissement, de chocs thermiques ou d'inclusions non metalliques. Tres dangereuses car elles peuvent se propager en service sous l'effet des contraintes cycliques.
- **Deformations geometriques.** Ecart par rapport a la geometrie nominale (gauchissement d'une aube, mauvaise concentricite). Causes par un retrait differentiel pendant la solidification ou par un demoulage premature.
- **Bavures (flash).** Excedents de matiere aux joints du moule, dus a un serrage insuffisant ou a une pression de coulee excessive.
- **Porosites de surface.** Petits pores visibles a la surface, resultant de gaz emprisonnes ou de retraits localises.



La figure 3.2 presente un panel representatif de ces anomalies en regard d'une piece conforme.



**Figure 3.2** – Exemples typiques de défauts comparés à une pièce conforme.

**Lecture de la figure 3.2.** Cette planche illustre la diversité morphologique des anomalies. La fissure (a) est linéaire et très locale, exigeant une résolution suffisante pour être détectée. La déformation (b) modifie la géométrie globale de la pièce, ce qui en fait un défaut détectable à basse fréquence spatiale. La bavure (c) est un excédent de matière localisée au bord. Enfin, la pièce conforme (d) sert de référence visuelle à notre modèle. **Cette diversité explique pourquoi un simple traitement d'image classique échoue : les défauts n'ont ni la même échelle, ni la même texture, ni la même localisation.** Seul un modèle d'apprentissage profond, capable d'apprendre simultanément plusieurs niveaux d'abstraction visuelle, peut espérer détecter cette variété de cas.

### 3.3 Objectif spécifique du projet

#### Formulation opérationnelle

Concevoir un **classifieur binaire** prenant en entrée une image numérique d'impeller et produisant en sortie une étiquette  $\hat{y} \in \{\text{def\_front}, \text{ok\_front}\}$  accompagnée d'un score de confiance  $p \in [0, 1]$ , avec :

- un *recall* sur la classe **def\_front** le plus élevé possible (priorité industrielle absolue) ;
- une *precision* suffisante pour éviter les fausses alertes (coûts de production) ;
- un temps d'inférence compatible avec la cadence de la ligne ( $< 100$  ms par pièce).

## 4. Acquisition et organisation des données

### 4.1 Choix du jeu de données

La construction d'un dataset spécifique aux impellers de fonderie aurait nécessité une campagne d'acquisition en usine, hors de portée dans le cadre de ce projet. Nous nous tournons donc vers un jeu de données public reconnu par la communauté : ***Casting Defect Detection*** [23], créé par Ravirajsinh Dabhi et publié sur Kaggle. Ce dataset présente plusieurs avantages :

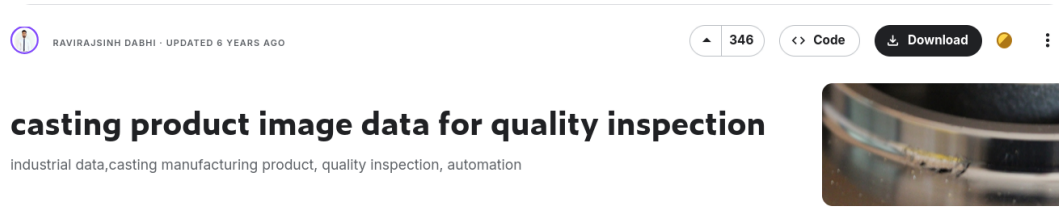
- il porte spécifiquement sur des *pièces de fonderie* (et non sur un domaine générique) ;
- les images sont prises dans des conditions *réalistes* d'éclairage et d'angle de vue ;
- il distingue explicitement deux classes (`def_front` et `ok_front`) représentant un cas typique de classification binaire industrielle ;
- il propose déjà une partition *train/validation/test*, garantissant la comparabilité avec d'autres travaux.

### 4.2 Description du dataset

Le dataset *Casting Defect Detection* contient des images en niveaux de gris ou couleur (selon la version) prises à la vue frontale (*front*) de pièces de fonderie. Il est organisé en deux classes binaires :

- `def_front` : pièces présentant un défaut visible (fissure, déformation, bavure, porosité).
- `ok_front` : pièces conformes aux spécifications.

La figure 4.1 présente un aperçu visuel du dataset.



**Figure 4.1** – Aperçu visuel du dataset *Casting Defect Detection* (Kaggle).

**Lecture de la figure 4.1.** On observe que les images partagent une mise en cadre standardisée : la pièce est centrée, prise sous un éclairage relativement uniforme et à une distance constante. Cette homogénéité est un atout pour l'apprentissage, mais elle constitue aussi une *limitation* : un modèle entraîné sur ces images ne généralisera pas nécessairement à des conditions d'usine différentes (angles obliques, éclairages non contrôlés). Cette considération sera reprise dans la discussion finale.

### 4.3 Repartition train / validation / test

Le dataset est partitionné en trois sous-ensembles disjoints selon la stratégie classique 80/10/10 :

| Ensemble   | Proportion | Role                                                 |
|------------|------------|------------------------------------------------------|
| Train      | 80 %       | Apprentissage des paramètres du modèle               |
| Validation | 10 %       | Reglage des hyperparamètres / <i>early stopping</i>  |
| Test       | 10 %       | Évaluation finale (jamais vu durant l'apprentissage) |

**Table 4.1** – Repartition train/validation/test du dataset.

**Justification.** Cette partition présente trois propriétés essentielles :

1. **Disjonction stricte** : aucune image n'appartient simultanément à deux ensembles. Cela garantit que les performances mesurées sur le test reflètent une vraie capacité de *generalisation*.
2. **Stratification** : la proportion des classes est préservée dans chaque sous-ensemble, évitant les biais d'échantillonnage.
3. **Reproductibilité** : la partition est fixée par l'auteur du dataset, ce qui permet la comparaison directe entre études utilisant ce même dataset.

## 4.4 Volumes et déséquilibre des classes

Le dataset compte environ **6 600 images d'entraînement**, et l'ensemble de test final regroupe **715 images** (453 defectueuses, 262 conformes). On observe ainsi un léger déséquilibre, avec environ **63 % de pièces defectueuses** contre **37 % de pièces conformes**. Ce déséquilibre, bien que modéré, justifie pleinement la mise en place d'un mécanisme de **ponderation des classes** qui sera détaillé dans le chapitre suivant.

### Pourquoi le déséquilibre est-il un problème ?

En l'absence de correction, un modèle entraîné sur un dataset déséquilibré tend à apprendre la *distribution prior* des classes plutôt que la séparation réelle entre les classes. Dans notre cas, un modèle « naïf » pourrait prédire systématiquement `def_front` et atteindre  $\sim 63\%$  d'*accuracy* sans rien avoir appris du contenu des images ! Ce piège est fondamental et doit être adressé explicitement.

## 5. Pretraitement des images

### 5.1 Philosophie generale du pipeline

La phase de pretraitement est l'une des plus determinantes en apprentissage profond applique a la vision : elle conditionne directement la *capacite de generalisation* du modele final. Notre pipeline est concu autour de quatre principes directeurs :

**Reproductibilite.** Toutes les sources d'aleatoire (Python, NumPy, frameworks) sont initialisees par une graine globale unique `SEED = 42`. Une experience peut donc etre reproduite a l'identique a partir du meme code et des memes donnees.

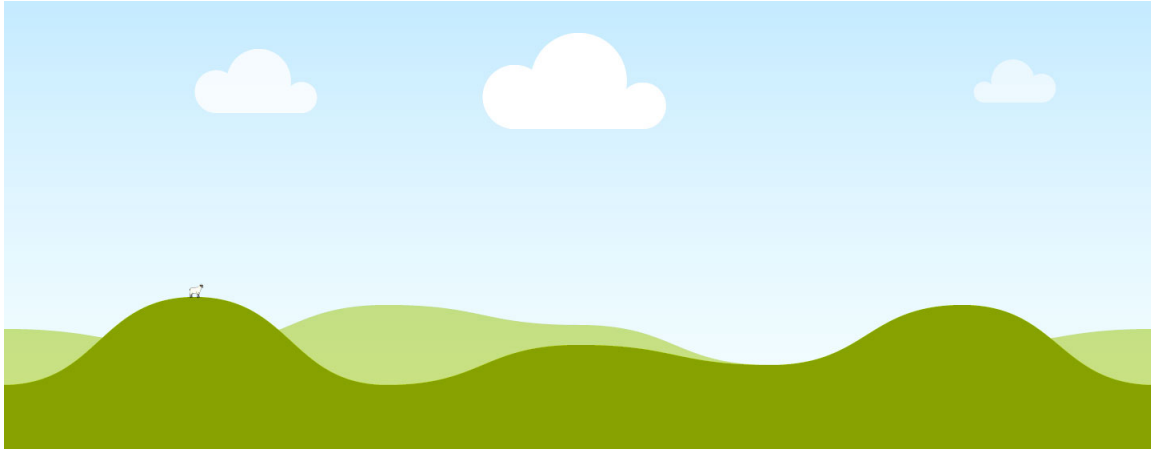
**Modularite.** Les transformations sont implementees comme des fonctions independantes, recombinaibles, reutilisables.

**Portabilite.** Le pipeline est implemente **en double version** (TensorFlow et PyTorch), validant ainsi la robustesse de la conception.

**Performance.** Utilisation systematique du parallelisme (`tf.data.AUTOTUNE` ou `num_workers`) et du *prefetching* pour saturer le Graphics Processing Unit (GPU).

### 5.2 Vue d'ensemble du pipeline TensorFlow (10 sections)

L'implementation TensorFlow est structuree en **dix sections fonctionnelles** successives, illustrees par le bandeau de la figure 5.1.



**Figure 5.1** – Bandeau de presentation du pipeline de pretraitement TensorFlow.

**Lecture de la figure 5.1.** Ce bandeau, utilise comme separateur visuel dans le notebook, marque l’entree dans la partie TENSORFLOW du pipeline. Il symbolise la chaine de transformations que chaque image traversera, depuis sa lecture brute sur le disque jusqu’a son arrivee comme tenseur normalise dans le modele.

Les dix sections de cette chaine sont les suivantes :

1. **Reproductibilite** : SEED = 42 (random, NumPy, TensorFlow).
2. **Detection GPU/CPU** : `tf.config.list_physical_devices()`.
3. **Color Balancing** : White Balance, Histogram Equalization, Channel Shuffle.
4. **Normalisation ImageNet** :  $\mu = [0.485, 0.456, 0.406]$ ,  $\sigma = [0.229, 0.224, 0.225]$ .
5. **Pipeline Train** : crop  $224 \times 224$ , flips, rotations, jitter.
6. **Pipeline Eval** : *center crop* deterministe + white balance.
7. **Chargement** : `image_dataset_from_directory` pour train/val/test.
8. **Class Weights** : equivalent du `WeightedRandomSampler` de PyTorch.
9. **Pipelines tf.data** : `.map()`  $\rightarrow$  `.batch()`  $\rightarrow$  `.prefetch(AUTOTUNE)`.
10. **Configuration** : export JSON, sanity check, visualisation.

### 5.2.1 Reproductibilite et detection du *device*

Pour garantir la reproductibilite, toutes les sources d’aleatoire sont fixees a une meme graine :

```

1 SEED = 42
2 random.seed(SEED)
3 np.random.seed(SEED)
4 tf.random.set_seed(SEED)
5
6 gpus = tf.config.list_physical_devices("GPU")
7 DEVICE = "GPU" if gpus else "CPU"
8 if gpus:
9     for gpu in gpus:
10         tf.config.experimental.set_memory_growth(gpu, True)
    
```

**Listing 5.1** – Initialisation reproductible et detection automatique du device.

**Commentaire.** L’option `set_memory_growth` est essentielle dans les environnements partages comme Google Colab : elle indique a TensorFlow de n’allouer la memoire GPU qu’au fur et a mesure des besoins, plutot que de reserver l’integralite de la VRAM des l’initialisation. Cela evite les erreurs `Out-Of-Memory` lors de l’execution simultanee de plusieurs notebooks.

**Hyperparametres globaux.** Le tableau 5.1 recapitule les constantes globales partagees par tous les notebooks du projet.

| Constante   | Valeur                        | Role                        |
|-------------|-------------------------------|-----------------------------|
| IMG_SIZE    | 224                           | Cote image (carree)         |
| BATCH_SIZE  | 32                            | Images par batch            |
| NUM_CLASSES | 2                             | <i>defect / ok_front</i>    |
| MEAN        | [0.485, 0.456, 0.406]         | Moyenne ImageNet (R/G/B)    |
| STD         | [0.229, 0.224, 0.225]         | Ecart-type ImageNet (R/G/B) |
| AUTOTUNE    | <code>tf.data.AUTOTUNE</code> | Parallelisme automatique    |

**Table 5.1** – Constantes globales partagees par tous les notebooks.

### 5.2.2 Color balancing : compatibilite *graph* TensorFlow

Les images industrielles souffrent souvent de *dominantes colorees* liees aux conditions d’eclairage (lumieres jaunatres ou bleuatres des ateliers). Pour rendre le modele robuste a ces variations, nous mettons en place trois transformations stochastiques de *color balancing* :

**Gray-World White Balance** ( $p = 0,5$ ). Sous l’hypothese *Gray-World* (la moyenne d’une scene naturelle tend vers le gris), on normalise chaque canal par sa moyenne, puis

on applique un facteur d'échelle borne dans  $[0,5 ; 2,0]$  pour eviter la sur-exposition. Implementee nativement avec `tf.cond`.

**Histogram Equalization** ( $p = 0,3$ ). Egalise l'histogramme canal par canal via `PIL.ImageOps.equalize` encapsulee dans `tf.py_function`. Fait ressortir les defauts subtils dans les zones peu contrastees.

**Channel Shuffle** ( $p = 0,15$ ). Permute aleatoirement l'ordre des canaux RGB (6 permutations possibles), pour rendre le modele *robuste aux derives de balance des couleurs* et l'empecher d'apprendre des correlations arbitraires entre teinte et classe.

```

1 mēâns = tf.rêdûcê_mēân(îmâgê, âxîs=[0, 1])
2 glôbâl_mēân = tf.rêdûcê_mēân(mēâns)
3 sçâlê = glôbâl_mēân / mēâns # (3,)
4 sçâlê = tf.çlîp_by_vâlûê(sçâlê, 0.5, 2.0)
5 rêtûrn tf.çlîp_by_vâlûê(îmâgê * sçâlê, 0., 1.)
    
```

**Listing 5.2** – Implementation du Gray-World White Balance.

**Commentaire.** Le clamp  $[0.5, 2.0]$  est crucial : sans lui, des images fortement saturees dans un canal pourraient produire des facteurs d'échelle extremes qui detruiraient l'information. Cette borne pragmatique est fixee empiriquement.

### 5.2.3 Normalisation ImageNet

La normalisation ImageNet aligne la distribution statistique de nos images sur celle attendue par les backbones pre-entraînes :

$$\hat{x} = \frac{x - \mu}{\sigma}, \quad \mu = \begin{pmatrix} 0.485 \\ 0.456 \\ 0.406 \end{pmatrix}, \quad \sigma = \begin{pmatrix} 0.229 \\ 0.224 \\ 0.225 \end{pmatrix} \quad (5.1)$$

L'equation (5.1) centre chaque canal sur la moyenne ImageNet et le reduit par son ecart-type. Cette etape est *indispensable* pour le *Transfer Learning* : les filtres pre-entraînes de EFFICIENTNETB0 ont ete optimises sur des entrees ainsi normalisees, et tout ecart de distribution entrainerait une chute drastique de performance.

### 5.2.4 Comparaison train vs eval

Une regle absolue en apprentissage profond : **l'augmentation de donnees ne s'applique qu'a l'entraînement**. Le tableau 5.2 compare les transformations appliquees dans chaque mode.



| Operation     | preprocess_train            | preprocess_eval              |
|---------------|-----------------------------|------------------------------|
| Resize        | $256 \times 256$            | $256 \times 256$             |
| Crop          | random $224 \times 224$     | center crop $224 \times 224$ |
| Flips         | H + V                       | —                            |
| Rotation      | $\pm 180^\circ$ (tfa)       | —                            |
| Color balance | stochastique                | deterministe ( $p = 1$ )     |
| Color jitter  | brightness/contrast/sat/hue | —                            |
| Grayscale     | $p = 0,1$                   | —                            |
| Normalisation | ImageNet                    | ImageNet                     |

 Table 5.2 – Comparaison des transformations *train* vs *eval*.

**Pourquoi cette dissymetrie ?** En entrainement, on cherche a *augmenter virtuellement* la taille du dataset par des transformations stochastiques, ce qui force le modele a apprendre des representations invariantes. En evaluation, au contraire, on veut une mesure *deterministe et reproductible* des performances : appliquer des transformations aleatoires biaiserait la mesure. Le seul *color balancing* eval applique est le White Balance avec  $p = 1$ , deterministe par construction.

### 5.2.5 Chargement et ponderation des classes

```

1 def _load_raw(directory, shuffle):
2     return tf.keras.utils.image_dataset_from_directory(
3         directory, labels="inferred", label_mode="int",
4         color_mode="rgb", batch_size=NONE,
5         image_size=(256, 256), shuffle=shuffle, seed=SEED,
6     )
7
8 raw_train = _load_raw(TRAIN_DIR, shuffle=True)
9 raw_val   = _load_raw(VAL_DIR,   shuffle=False)
10 raw_test  = _load_raw(TEST_DIR,  shuffle=False)
11
12 all_labels = np.array([lbl.numpy() for _, lbl in raw_train])
13 class_counts = np.bincount(all_labels, minlength=NUM_CLASSES)
14 total       = len(all_labels)
15 class_weights = {
16     i: float(total) / (NUM_CLASSES * int(count))
17     for i, count in enumerate(class_counts)
18 }
    
```

 Listing 5.3 – Chargement `tf.data` et calcul des poids de classes.

La ponderation suit la formule classique *inverse frequency weighting* :

$$w_i = \frac{N_{\text{total}}}{N_{\text{classes}} \cdot n_i} \quad (5.2)$$

Cette formule donne un poids superieur a la classe minoritaire (`ok_front`) afin que le modele consacre proportionnellement plus d'attention aux echantillons rares lors du calcul de la fonction de perte.

### 5.2.6 Composition finale des datasets

```

1  train_dâtasêt = (râw_train
2      .mâp(prêprôçêss_train, nûm_pârâllêl_çâlls=AUTOTUNE)
3      .bâtçh(BATCH_SIZE, drôp_rêmâindêr=Trûê)
4      .prêfêçh(AUTOTUNE))
5
6  val_dâtasêt = (râw_val
7      .mâp(prêprôçêss_êvâl, nûm_pârâllêl_çâlls=AUTOTUNE)
8      .bâtçh(BATCH_SIZE).prêfêçh(AUTOTUNE))
9
10 test_dâtasêt = (râw_têst
11     .mâp(prêprôçêss_êvâl, nûm_pârâllêl_çâlls=AUTOTUNE)
12     .bâtçh(BATCH_SIZE).prêfêçh(AUTOTUNE))
    
```

**Listing 5.4** – Composition finale : `.map()` -> `.batch()` -> `.prefetch()`.

**Commentaire.** L'ordre `.map()` → `.batch()` → `.prefetch()` est crucial. `.map()` applique la transformation a chaque image *individuellement* en parallele, `.batch()` regroupe ensuite les images traitees en batches, et `.prefetch(AUTOTUNE)` permet au CPU de preparer le prochain batch pendant que le GPU traite l'actuel — realisant ainsi un pipelining qui maximise l'utilisation des deux unites de calcul.

### 5.2.7 Configuration centralisee et utilitaires

```

1  CONFIG = {
2      "îmg_sîzê"      : 224,
3      "bâtçh_sîzê"    : 32,
4      "nûm_çlâssês"   : 2,
5      "çlâss_nâmês"   : ["dêfêçt", "ôk_frônt"],
6      "mêân"          : [0.485, 0.456, 0.406],
7      "std"            : [0.229, 0.224, 0.225],
8      "çlâss_wêîghts" : {"0": 1.12, "1": 0.90},
9      "dêvîçê"        : "GPU"
10 }
    
```

**Listing 5.5** – Export centralise de la configuration au format JSON.

Quatre utilitaires complètent le module `preprocessing_tf` :

- `sanity_check()` — affiche *shape*, min/max/mean d'un batch.
- `show_batch(ds, n=8)` — visualise *n* images avec leur label.
- `plot_class_distribution()` — histogramme *defect* vs *ok\_front*.
- `compute_mean_std(ds, n=20)` — moyenne et ecart-type empiriques.

La figure 5.2 regroupe les sorties typiques de ces quatre utilitaires.



**Figure 5.2** – Sorties des utilitaires de controle du pipeline.

**Lecture de la figure 5.2.** La sous-figure (a) confirme la coherence des shapes du batch ( $32 \times 224 \times 224 \times 3$ ) ainsi que les statistiques pixel apres normalisation (min  $\approx -2.1$ , max  $\approx 2.6$ ,  $\mu \approx 0.3$ ) — valeurs coherentes avec la normalisation ImageNet, ce qui valide la chaine de pretraitement. La sous-figure (b) presente un batch de 8 images apres pretraitement complet : on y voit des pieces dans des orientations variees, traduisant l'effet des rotations et des flips. La sous-figure (c) confirme visuellement le desequilibre annonce des classes : la barre rouge (**defect**) est systematiquement superieure a la verte (**ok\_front**) dans les trois splits, ce qui justifie a posteriori le mecanisme de *class weighting* mis en place.

## 5.3 Pipeline equivalent en PyTorch

Le pipeline PYTORCH reprend les memes etapes que la version TensorFlow, mais s'appuie sur les abstractions natives de `torchvision.transforms` et un `DataLoader` avec `WeightedRandomSampler`.

### 5.3.1 Reproductibilite et device

```

1 SEED = 42
2 random.seed(SEED); np.random.seed(SEED)
3 torch.manual_seed(SEED); torch.cuda.manual_seed_all(SEED)
4 torch.backends.cudnn.deterministic = True
5 torch.backends.cudnn.benchmark = False
6
    
```

```
7 DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

**Listing 5.6** – Reproductibilité et détection automatique du device en PyTorch.

**Commentaire.** Les options `torch.backends.cudnn.deterministic = True` et `torch.backends.cudnn.benchmark = False` forcent CUDA à utiliser des algorithmes déterministes, au prix d’une légère perte de performance. C’est un *compromis assumé* en faveur de la reproductibilité, particulièrement important en contexte académique ou industriel où les résultats doivent pouvoir être reproduits à l’identique.

### 5.3.2 Color balancing personnalisée

PyTorch ne propose pas nativement les transformations de *color balancing* dont nous avons besoin ; nous les implémentons donc comme des classes héritant de `torch.nn.Module` :

- **AutoWhiteBalance** ( $p = 0,5$ ) — hypothèse *Gray-World* pour corriger les dominantes colorées des caméras industrielles.
- **HistogramEqualization** ( $p = 0,3$ ) — fait ressortir les défauts subtils dans les zones peu contrastées.
- **ChannelShuffle** ( $p = 0,15$ ) — permute les canaux RGB pour rendre le modèle indépendant de la teinte de surface.

### 5.3.3 Pipeline complet d’augmentations

Les transformations appliquées au *train* se composent comme suit :

1. `Resize(256, 256)` — uniformisation de la taille d’entrée.
2. `RandomCrop(224)` — recadrage aléatoire  $224 \times 224$  pour introduire de la variabilité spatiale.
3. `RandomHorizontalFlip(0.5)` et `RandomVerticalFlip(0.5)` — les images industrielles n’ont pas d’orientation privilégiée.
4. `RandomRotation(180)` — rotations jusqu’à  $\pm 180^\circ$  : invariance rotationnelle complète.
5. `AutoWhiteBalance(0.5)`, `HistogramEqualization(0.3)`, `ChannelShuffle(0.15)` — robustesse couleur.

6. `ColorJitter(0.2, 0.3, 0.1, 0.02)` — variations *brightness, contrast, saturation, hue*.
7. `RandomAffine(translate=(0.05, 0.05))` — translations  $\pm 5\%$  : robustesse aux decalages de cadrage.
8. `RandomGrayscale(0.1)` — 10% des images converties en niveaux de gris.
9. `GaussianBlur(3)` — flou léger : robustesse au bruit de capteur.
10. `ToTensor()` et `Normalize( $\mu, \sigma$ )` (ImageNet).

### 5.3.4 Gestion du desequilibre des classes

Le dataset presente un leger desequilibre ( $\sim 65\%$  `def_front` vs  $\sim 35\%$  `ok_front`). Sans correction, le modele apprendrait a toujours predire la classe majoritaire. Nous utilisons un `WeightedRandomSampler` :

```

1  glâss_çôûnts    = tôrçh.bîñçôûnt(târgêts)
2  glâss_wêights   = 1.0 / glâss_çôûnts
3  sâmplê_wêights  = glâss_wêights[târgêts]
4  sâmplêr = WêightêdRândômSâmplêr(
5      sâmplê_wêights, nûm_sâmplês=lên(sâmplê_wêights),
6      rêplâçêment=True
7  )
    
```

**Listing 5.7** – Ponderation inverse-frequence des echantillons.

**Commentaire.** L’option `replacement=True` est essentielle : sans elle, les echantillons de la classe minoritaire seraient epuises avant ceux de la classe majoritaire. Avec replacement, chaque epoque tire  $N$  echantillons au sort selon la distribution ponderee, garantissant une exposition equilibree du modele aux deux classes.

### 5.3.5 DataLoaders et utilitaires

Le tableau 5.3 resume la configuration des trois `DataLoaders`.

| Loader                    | batch_size | shuffle | Sampler                     | pin_memory |
|---------------------------|------------|---------|-----------------------------|------------|
| <code>train_loader</code> | 32         | —       | <code>WeightedRandom</code> | si CUDA    |
| <code>val_loader</code>   | 32         | False   | —                           | si CUDA    |
| <code>test_loader</code>  | 32         | False   | —                           | si CUDA    |

**Table 5.3** – Configuration des trois `DataLoaders` PyTorch.

**Pourquoi `pin_memory=True` ?** L'option `pin_memory` alloue les tenseurs dans une zone mémoire CPU « pinnée », accélérant le transfert vers le GPU via DMA asynchrone. Le gain peut atteindre 30% sur les transferts CPU-GPU, ce qui est significatif lorsque le bottleneck du training est le chargement des données.

## 6. Modelisation et experimentations

Dans ce chapitre, nous developpons et comparons **trois architectures de classification** suivant une demarche progressive : (i) un Perceptron Multi-Couches (MLP) servant de baseline, (ii) un Reseau de Neurones Convolutif (CNN) entraine de zero, et (iii) un modele de *Transfer Learning* fonde sur EFFICIENTNETB0. Cette demarche permet de mettre en evidence, de maniere empirique et pedagogique, l'apport specifique de chaque famille architecturale.

### 6.1 Modele 1 : Perceptron Multi-Couches (MLP)

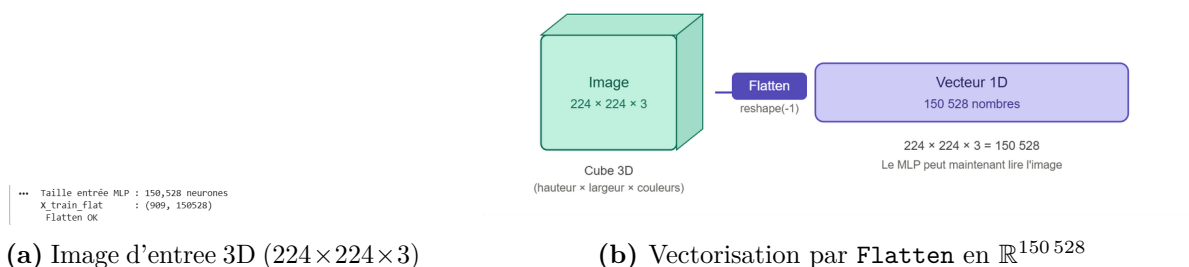
#### 6.1.1 Motivation

Le MLP constitue le modele de neurones le plus elementaire capable d'apprentissage supervise non-lineaire. Il sert ici de **baseline** : a la fois comme reference de performance *minimale* et comme pretexte pedagogique pour mettre en evidence les limites des reseaux *denses* face aux donnees structurees comme les images.

#### 6.1.2 Representation des donnees : le Flatten

Les images d'entree sont des tenseurs  $224 \times 224 \times 3$  (hauteur  $\times$  largeur  $\times$  canaux). Or, un MLP ne sait traiter que des *vecteurs unidimensionnels* : il faut donc *aplatir* l'image, operation realisee par la couche **Flatten** :

$$(224 \times 224 \times 3) \longrightarrow \mathbb{R}^{150\,528} \quad (6.1)$$



**Figure 6.1** – Aplatissement (*Flatten*) d'une image en vecteur 1D.

**Lecture de la figure 6.1.** La sous-figure (a) represente l'image dans sa structure naturelle 3D, ou la *proximite spatiale* entre pixels traduit une realite physique : deux pixels

voisins horizontalement ou verticalement appartiennent typiquement a la meme region (meme aube, meme bord). La sous-figure (b) illustre la transformation **Flatten**, qui *linearise* ces 150 528 valeurs sans aucune notion d'adjacence. **C'est la limite fondamentale du MLP applique aux images** : deux pixels voisins en 2D peuvent se retrouver tres eloignes dans le vecteur 1D, ce qui empeche le reseau de tirer parti de la structure spatiale.

### 6.1.3 Architecture du MLP

#### Architecture du MLP

**Entree** : 150 528 neurones

→ Dense(512) + BatchNorm + ReLU + Dropout(0.4)

→ Dense(256) + BatchNorm + ReLU + Dropout(0.4)

→ Dense(128) + BatchNorm + ReLU + Dropout(0.3)

→ Dense(2) + **Softmax** (→ def\_front / ok\_front)

La figure 6.2 represente schematiquement cette architecture.

| Model: "MLP"                             |              |            |
|------------------------------------------|--------------|------------|
| Layer (type)                             | Output Shape | Param #    |
| dense (Dense)                            | (None, 512)  | 77,070,848 |
| batch_normalization                      | (None, 512)  | 2,048      |
| activation (ReLU)                        | (None, 512)  | 0          |
| dropout (Dropout 0.4)                    | (None, 512)  | 0          |
| dense_1 (Dense)                          | (None, 256)  | 131,328    |
| batch_normalization_1                    | (None, 256)  | 1,024      |
| activation_1 + dropout_1 (0.4)           | (None, 256)  | 0          |
| dense_2 (Dense)                          | (None, 128)  | 32,896     |
| batch_normalization_2                    | (None, 128)  | 512        |
| activation_2 + dropout_2 (0.3)           | (None, 128)  | 0          |
| dense_3 (Dense + Softmax)                | (None, 2)    | 258        |
| Total params: 77,238,914 (294.64 MB)     |              |            |
| Trainable params: 77,237,122 (294.64 MB) |              |            |
| Non-trainable: 1,792 (7.00 KB)           |              |            |

**Figure 6.2** – Schema global de l'architecture MLP.

**Lecture de la figure 6.2.** Le reseau enchainne trois blocs denses dont la dimension decroit progressivement (512 → 256 → 128). Cette decroissance, classique en architecture neuronale, joue le role d'un *compresseur d'information* : le reseau doit progressivement *filtrer* l'information pertinente parmi les 150 528 features brutes. La sortie est une couche dense de 2 neurones suivie d'une fonction *Softmax* qui produit une distribution de probabilite sur les deux classes.



### 6.1.4 Mecanismes de regularisation

Trois mecanismes de regularisation sont integres pour limiter le sur-apprentissage :

- **Batch Normalization** [8] : normalise les activations intermediaires sur le batch, stabilisant et accelerant l'apprentissage. Mathematiquement, pour une activation  $x$  d'un mini-batch :

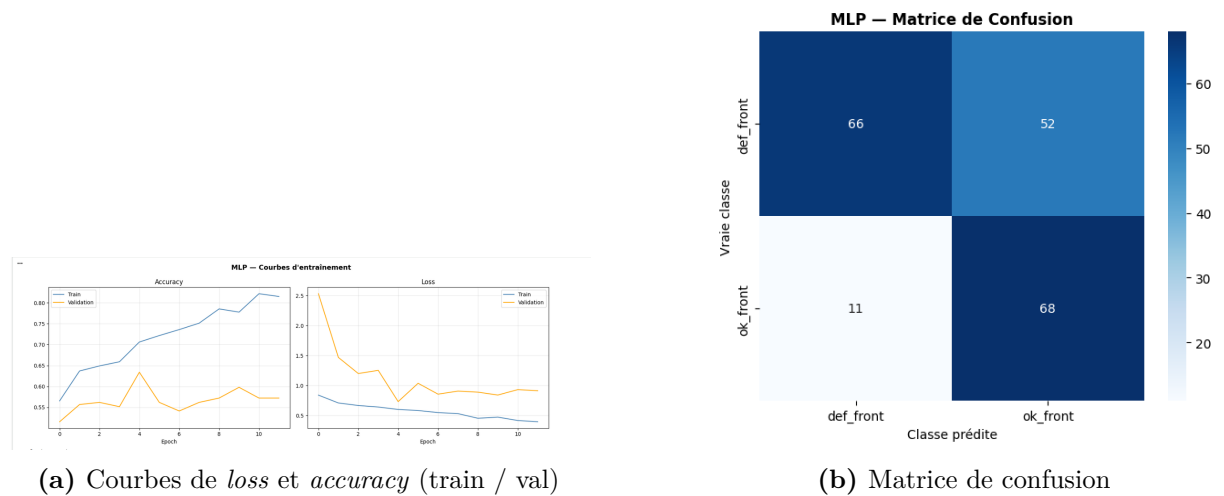
$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma\hat{x} + \beta \quad (6.2)$$

ou  $\gamma, \beta$  sont des parametres appris.

- **Dropout** [9] : a chaque iteration d'entrainement, une fraction  $p$  des neurones est aleatoirement desactivee. Cela force le reseau a apprendre des representations *redundantes* et l'empeche de trop dependre d'un sous-ensemble de neurones.
- **Early Stopping (patience = 7)** : arret automatique de l'entrainement si la perte de validation cesse de diminuer pendant 7 epoques consecutives. Sauvegarde le meilleur modele observe.
- **ReduceLROnPlateau** : divise le *learning rate* par 2 si la perte stagne, permettant un *fine-tuning* progressif.

### 6.1.5 Resultats du MLP

Le modele est entraine pendant un maximum de 30 epoques avec optimiseur Adam (LR initial = 1e-3) et perte *categorical crossentropy*. Les courbes d'apprentissage et la matrice de confusion finale sont presentees figure 6.3.



**Figure 6.3** – Performance du MLP sur le jeu de test.

**Lecture de la figure 6.3.** La sous-figure (a) montre que l'*accuracy* de train et de validation evoluent de maniere proche, plafonnant rapidement autour de  $\sim 70\%$ . La perte (*loss*) decroit puis se stabilise. L'absence d'ecart marque entre train et validation indique **l'absence de sur-apprentissage** : le probleme du MLP n'est pas qu'il apprend par cœur ses donnees, mais qu'il n'est *intrinsequement pas capable* d'apprendre une fonction de classification suffisamment expressive sur ce type d'entree. La sous-figure (b) confirme cette analyse : la matrice de confusion montre des erreurs symetriquement reparties sur les deux classes (52 `def_front` mal classees, 11 `ok_front` mal classees), traduisant une difficulte structurelle plutot qu'un biais.

Le tableau 6.1 synthetise les metriques finales sur le test.

| Metrique          | Valeur                           |
|-------------------|----------------------------------|
| Test Loss         | 0,7587                           |
| Test Accuracy     | 68,02 %                          |
| Precision         | 74,07 %                          |
| Recall            | 68,02 %                          |
| F1-Score          | 67,95 %                          |
| Epochs entrainees | 12/30 (early stopping declenche) |

**Table 6.1** – Resultats du MLP sur l'ensemble de test.

### 6.1.6 Analyse critique du MLP

**Points forts.** Convergence rapide (12 epoques), modele simple a comprendre et a entrainer, baseline honnete pour quantifier l'apport des architectures suivantes.

**Limites.** La couche `Flatten` **detruit la structure spatiale** : les pixels voisins perdent leur lien, le modele ne peut pas apprendre de *textures* ni de *contours*. De plus, le nombre colossal de parametres dans la premiere couche dense ( $150\,528 \times 512 \approx 77$  millions!) est totalement disproportionne par rapport a la taille du dataset, augmentant le risque de sur-apprentissage et alourdissant inutilement le modele.

⇒ Ces limitations motivent le passage a un modele exploitant explicitement la structure 2D de l'image : le **Reseau de Neurones Convolutif (CNN)**.

## 6.2 Modele 2 : CNN entraine de zero

### 6.2.1 Principe et fondements theoriques

Un **Reseau de Neurons Convolutif (CNN)** [1] est une architecture concue speciquement pour exploiter la structure spatiale des images. Il repose sur trois operations fondamentales :

1. **La convolution** : un filtre (ou *kernel*) de taille reduite (typiquement  $3 \times 3$ ) glisse sur l'image et calcule un produit scalaire local. Le meme filtre est applique partout (*poids partages*), ce qui reduit drastiquement le nombre de parametres et confere au reseau une *invariance par translation*.
2. **Le pooling** : reduction spatiale (*max pooling* ou *average pooling*) qui resume une region locale en un unique pixel, augmentant le champ receptif et reduisant la dimension.
3. **Les non-linearites** : typiquement ReLU (Rectified Linear Unit), qui introduit la capacite de modeliser des fonctions non-lineaires complexes.

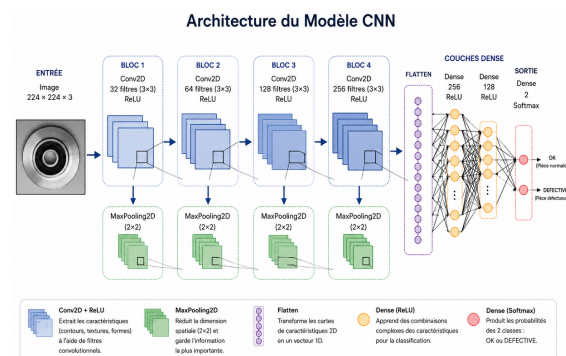
L'empilement de ces operations construit une *hierarchie de representations* : les premieres couches detectent des contours et des couleurs, les couches intermediaires des textures et des motifs, les couches profondes des concepts semantiques.

### 6.2.2 Architecture du CNN propose

Notre CNN enchaîne plusieurs blocs convolutifs, chacun compose d'une couche Conv2D, suivie de BatchNormalization, d'une activation ReLU et d'une couche MaxPooling2D. La figure 6.4 illustre l'architecture globale.

```
model_cnn = models.Sequential([
    layers.Input(shape=(224, 224, 3)),
    # Bloc 1
    layers.Conv2D(32, (3,3), activation='relu',padding='same'),
    layers.MaxPooling2D((2,2)),
    #Bloc 2
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    #Bloc 3
    layers.Conv2D(128, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    #Bloc 4
    layers.Conv2D(256, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    #Flatten
    layers.Flatten(),
    # Dense layers
    layers.Dense(256, activation='relu'),
    layers.Dense(128, activation='relu'),
    # Output
    layers.Dense(2, activation='softmax')])
```

(a) Bloc convolutif elementaire



(b) Vue d'ensemble du CNN

Figure 6.4 – Architecture du CNN entraine de zero.

**Lecture de la figure 6.4.** La sous-figure (a) présente la *cellule élémentaire* répétée dans le réseau : convolution, batch norm, ReLU, pooling. C'est l'unité atomique d'extraction de caractéristiques. La sous-figure (b) montre l'agencement global : plusieurs blocs convolutifs en série, dont la profondeur (nombre de filtres) augmente progressivement, suivis d'un *Global Average Pooling* et d'une tête de classification dense. Cette architecture pyramidale est typique des CNN et reflète la hiérarchie d'abstraction visuelle évoquée plus haut.

### 6.2.3 Compilation et régularisation

Le modèle est compilé avec :

- **Optimiseur** : Adam [10] avec `learning_rate=1e-3`.
- **Fonction de perte** : *categorical\_crossentropy* (formulation pour 2 classes one-hot).
- **Métriques** : accuracy.

Plusieurs mécanismes de régularisation sont intégrés :

```
model_cnn.compile(optimizer="adam",
                  loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])
```

**Figure 6.5** – Compilation et configuration du CNN avec callbacks de régularisation.

**Lecture de la figure 6.5.** La compilation Keras révèle trois mécanismes de régularisation simultanés : l'*Early Stopping* (arrêt si la `val_loss` ne s'améliore plus), la *Batch Normalization* (déjà décrite, intégrée dans chaque bloc convolutif) et le *Dropout* (désactivation aléatoire de neurones). La combinaison de ces trois mécanismes correspond aux meilleures pratiques actuelles pour entraîner un CNN sur un dataset de taille modérée.

### 6.2.4 Entraînement

Le CNN est entraîné pendant 30 époques maximum, avec batch de taille 32. La figure 6.6 montre la boucle d'entraînement et un extrait des logs Keras.

```
model_cnn_reg = models.Sequential([
    #Input
    layers.Input(shape=(224, 224, 3)),
    #Block 1
    layers.Conv2D(32, (3,3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    #Block 2
    layers.Conv2D(64, (3,3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    #Block 3
    layers.Conv2D(128, (3,3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    #Block 4
    layers.Conv2D(256, (3,3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    #Flatten
    layers.Flatten(),
    #Dense layers
    layers.Dense(256),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dropout(0.5),
    layers.Dense(128),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dropout(0.5),
    # Output
    layers.Dense(2, activation='softmax')])
```

 (a) Boucle d'entrainement `model.fit()`

```
early_stop_cnn = callbacks.EarlyStopping(
    monitor="val_loss",
    patience=3,
    restore_best_weights=True
)

history_cnn_reg = model_cnn_reg.fit(
    train_loader,
    validation_data=val_loader,
    epochs=20,
    callbacks=[early_stop_cnn],
    verbose=1
)
```

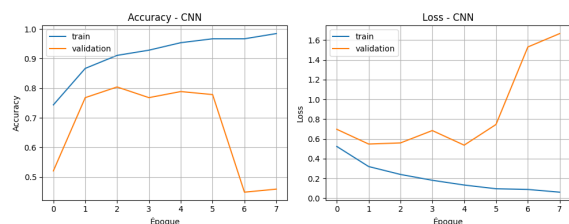
(b) Extrait des logs Keras

Figure 6.6 – Entrainement du CNN sur 30 epoques.

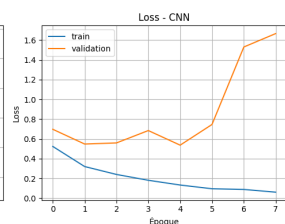
**Lecture de la figure 6.6.** La sous-figure (a) montre l'appel canonique `model.fit()` avec ses parametres essentiels : dataset d'entrainement, dataset de validation, nombre d'epoques, callbacks. La sous-figure (b) presente un extrait des logs : on y voit les progres epoque par epoque, avec accuracy de train et accuracy de validation. Ces logs sont essentiels pour le diagnostic en temps reel du comportement du modele.

### 6.2.5 Evaluation et diagnostic d'overfitting

Les courbes d'apprentissage du CNN sans data augmentation revelent un comportement caracteristique illustre figure 6.7.



(a) Accuracy train (bleue) vs validation (orange)



```
test_loss_cnn_reg, test_acc_cnn_reg = model_cnn_reg.evaluate(
    test_loader,
    verbose=0
)

print("Test Loss:", test_loss_cnn_reg)
print("Test Accuracy:", test_acc_cnn_reg)

Test Loss: 0.6228603720664978
Test Accuracy: 0.8020304441452026
```

(b) Loss train (bleue) vs validation (orange)

Figure 6.7 – Courbes d'entrainement du CNN avant data augmentation : signe clair de sur-apprentissage.

**Lecture de la figure 6.7 : diagnostic d'overfitting.**

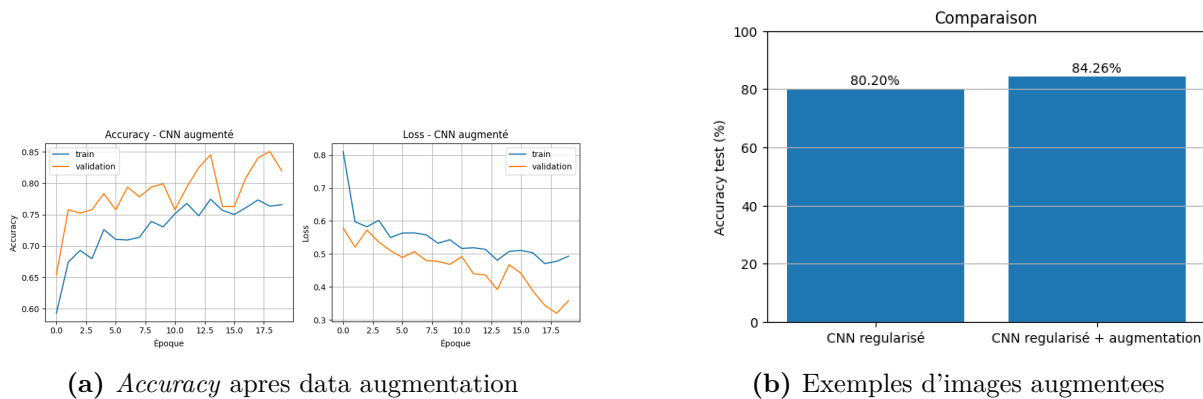
- *Accuracy de train* : monte rapidement jusqu'a  $\sim 99\%$   $\rightarrow$  le modele apprend tres bien les donnees d'entrainement.
- *Accuracy de validation* : monte d'abord en parallele, puis **plafonne** et commence meme a **decroitre** dans les dernieres epoques.

- *Loss de train* : décroît regulierement.
- *Loss de validation* : décroît d'abord, puis **remonte** a partir de l'époque  $\sim 15$ .

⇒ **Diagnostic clair de sur-apprentissage (overfitting)** : le modele memorise les donnees d'entraînement plutôt que d'apprendre des regles generalisables. La divergence entre courbes de train et de validation est le marqueur classique de ce phenomene.

### 6.2.6 Regularisation par augmentation des donnees

Pour combattre l'overfitting, nous renforcons la *data augmentation* en ajoutant des transformations supplementaires (rotations, flips, jitter colore, zoom). Ces transformations *augmentent virtuellement* la taille du dataset et forcent le modele a apprendre des representations invariantes a ces variations. Les resultats apres augmentation sont presentes figure 6.8.



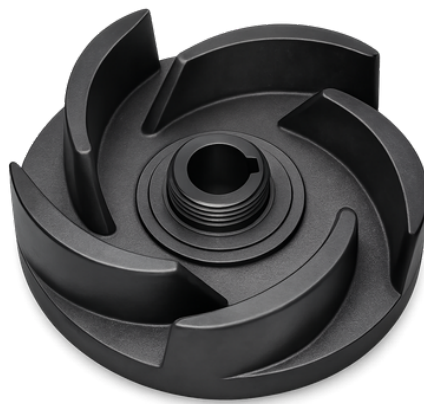
**Figure 6.8** – Effet de la *data augmentation* sur l'apprentissage.

**Lecture de la figure 6.8.** La sous-figure (a) montre des courbes *accuracy* train et validation desormais beaucoup plus proches l'une de l'autre, avec une validation qui suit la train sans plonger : **l'overfitting est largement resorbe**. La sous-figure (b) illustre quelques transformations appliquees au meme echantillon : rotations diverses, flips, variations de luminosite. Ces « copies legerement perturbees » multiplient virtuellement la diversite des exemples vus par le reseau.

## 6.2.7 Conclusion intermédiaire sur le CNN

### Bilan du CNN

La data augmentation corrige efficacement l'overfitting et fournit un modèle plus robuste. Cependant, atteindre 99% de précision avec un CNN *entraîne de zero* nécessiterait beaucoup plus de données ( $> 10^5$  images typiquement). Sur un dataset de  $\sim 6\,600$  images, le CNN plafonne aux alentours de 95% d'accuracy. C'est précisément la situation où le **Transfer Learning** apporte un bénéfice décisif : au lieu de partir d'un réseau aux poids aléatoires, on démarre d'un réseau pré-entraîné sur ImageNet, qui contient déjà des représentations visuelles riches et transférables.



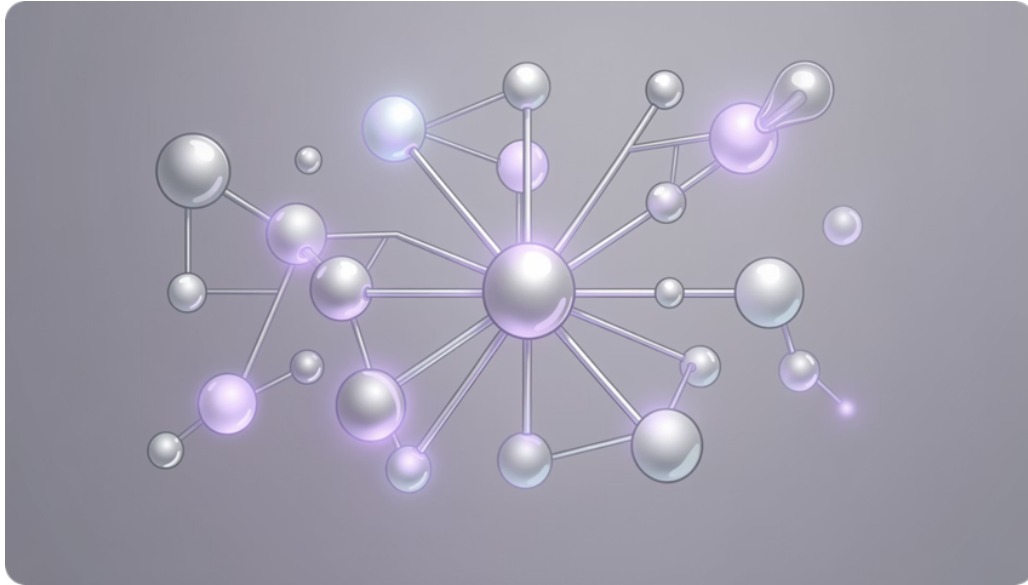
**Figure 6.9** – Synthèse visuelle des résultats du CNN.

**Lecture de la figure 6.9.** Ce visuel résume schématiquement le bilan du modèle CNN entraîne de zero : bonne performance, structure spatiale exploitée, mais limitée par la taille du dataset. C'est le point de bascule qui motive l'introduction du Transfer Learning.

## 6.3 Modèle 3 : Transfer Learning avec EfficientNetB0

### 6.3.1 Principe du Transfer Learning

Le *Transfer Learning* (Transfer Learning (apprentissage par transfert) (TL), apprentissage par transfert) consiste à **reutiliser un réseau de neurones déjà entraîné** sur une grande base d'images — ici ImageNet, qui contient  $\sim 1,2$  million d'images réparties sur 1 000 classes — pour résoudre un nouveau problème plus spécifique. Ce paradigme exploite le fait que les premières couches d'un CNN apprennent des concepts visuels *généralisés* (contours, textures, formes) qui sont *transférables* à presque toutes les tâches d'images naturelles.



**Figure 6.10** – Principe general du Transfer Learning : reutilisation des features d'un modele pre-entraine.

**Lecture de la figure 6.10.** Le schema illustre le mecanisme de transfert : a gauche, un reseau profond pre-entraine sur ImageNet a deja appris a extraire des features riches sur 1 000 categories de la vie courante. A droite, ce meme reseau (ou ses premieres couches) est reutilise comme *extracteur de features* sur notre probleme cible (detection de défauts), ne necessitant que l'apprentissage d'une nouvelle tete de classification specifique. Le gain est double : convergence plus rapide *et* meilleure generalisation, car le reseau beneficie de millions d'images d'entrainement initiales.

### 6.3.2 Strategie en deux phases

Notre strategie de Transfer Learning suit le schema canonique en **deux phases successives** :

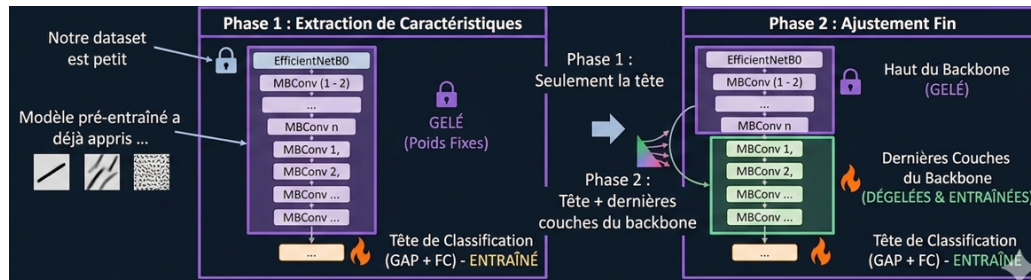
| Phase                             | Ce que l'on entraine                                                                    |
|-----------------------------------|-----------------------------------------------------------------------------------------|
| 1. Extraction de caracteristiques | <b>Tete de classification uniquement</b> : le <i>backbone</i> pre-entraine est gele.    |
| 2. <i>Fine-tuning</i>             | Tete + dernieres couches du <i>backbone</i> , avec un <i>learning rate</i> tres faible. |

**Table 6.2** – Strategie de Transfer Learning a deux etages.

**Justification.**



- Notre dataset est relativement petit ( $\sim 6\,600$  images d'entraînement) : entrainer un grand reseau de zero provoquerait du sur-apprentissage.
- Les caracteristiques visuelles generiques apprises sur ImageNet (contours, textures, formes) sont transferables a presque toute tache d'image.
- La phase 1 evite de « casser » les filtres pre-entraines ; la phase 2 les ajuste finement a notre tache specifique.

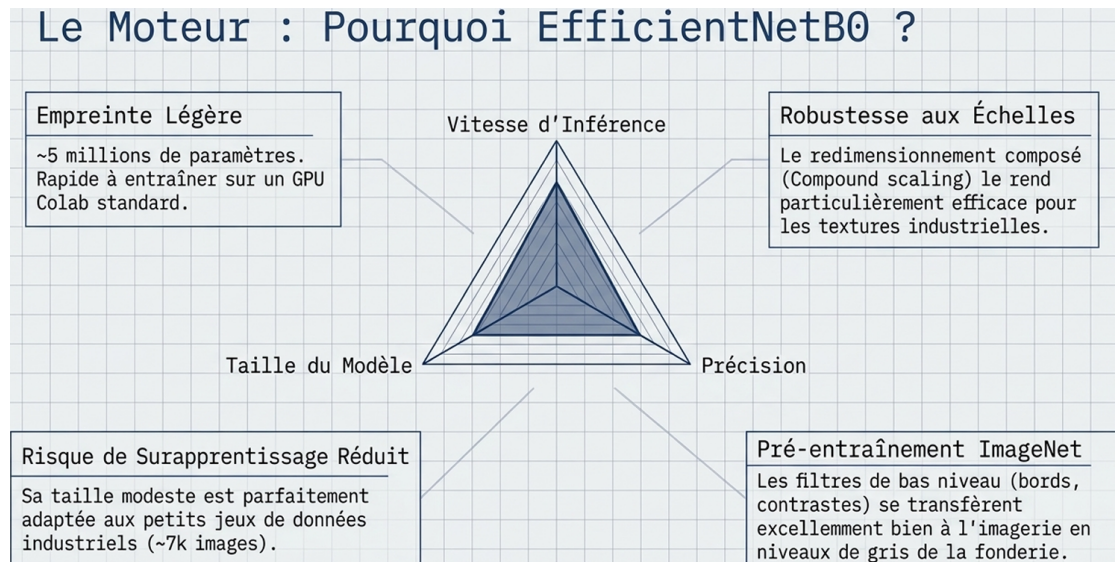


**Figure 6.11** – Reutilisation du *backbone* pre-entraîne : la tete originale (1 000 classes ImageNet) est remplacée par une tete binaire.

**Lecture de la figure 6.11.** Le schema montre la transformation opereee sur le reseau pre-entraîne : on **conserve** les couches convolutives (qui constituent l'extracteur de features generique) et on **remplace** la tete originale, prevue pour 1 000 classes ImageNet, par une nouvelle tete adaptee a notre probleme binaire. Le *backbone* reste fige durant la phase 1.

### 6.3.3 Architecture choisie : EfficientNetB0

EFFICIENTNET [6] est une famille d'architectures developpee par Google en 2019, dont la specificite est de **coupler haute performance et faible cout de calcul** grace au *compound scaling* deja decrit dans l'etat de l'art (chapitre 2). Le modele EFFICIENTNETB0, a la base de la famille, compte 5,3 millions de parametres et atteint 77,3% de top-1 accuracy sur ImageNet — excellent compromis pour le Transfer Learning sur dataset modere.

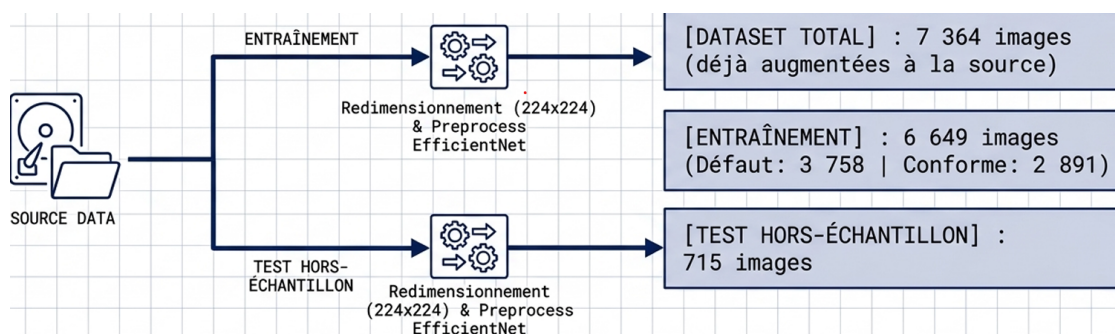


**Figure 6.12** – Architecture détaillée d'EfficientNetB0 (blocs MBConv).

**Lecture de la figure 6.12.** L'architecture s'articule autour de blocs MBConv (*Mobile Inverted Bottleneck*) avec mécanismes *Squeeze-and-Excitation*. La sequence de blocs alterne diverses tailles de filtres ( $3 \times 3$ ,  $5 \times 5$ ) et facteurs d'expansion. La fin du backbone produit un tenseur de features dimensionné  $7 \times 7 \times 1280$ , qui sera ensuite condensé par un *Global Average Pooling* avant d'entrer dans la tête de classification.

### 6.3.4 Source des données et preparation

Le dataset utilise pour cette experience est la **version augmentée** fournie directement par l'auteur sur Kaggle, déjà adaptée aux contraintes industrielles (variations d'éclairage, angles, textures de surface). Notre pretraitement se limite donc au redimensionnement et à la normalisation requise par EfficientNetB0.



**Figure 6.13** – Dataset *Casting Defect Detection* dans sa version augmentée fournie sur Kaggle.

**Lecture de la figure 6.13.** On observe la diversité des exemples fournis, avec variations d'angle, de zoom et de luminosité. Cette diversité est précieuse car elle permet au modèle d'apprendre des représentations robustes *sans qu'il nous soit nécessaire d'ajouter une*

*augmentation manuelle supplémentaire.* Le travail de l’auteur du dataset constitue donc une vraie valeur ajoutée.

**Chargement automatique.** Keras lit automatiquement l’arborescence `train/ok_front/` et `train/def_front/`, redimensionne en  $224 \times 224$  et attribue un label binaire en conséquence. La fonction `preprocess_input` d’EfficientNet transforme ensuite les valeurs  $[0, 255]$  vers  $[-1, 1]$  (centrage attendu par le backbone).

## Préparation des données

### a) Chargement depuis les dossiers

```
train_ds_raw = keras.utils.image_dataset_from_directory(
    TRAIN_DIR,
    labels='inferred',           # Les labels sont déduits du nom des dossiers
    label_mode='binary',        # 0 = bonne pièce, 1 = défectueuse
    image_size=(224, 224),      # redimensionne-la en un carré de 224 pixels de côté
    batch_size=32,
    shuffle=True,               # oblige à rester attentive aux détails plutôt qu'à l'ordre des images.
    validation_split=0.15,
    subset='training',
)
```

Keras lit automatiquement les images depuis l’arborescence `train/ok_front/` et `train/def_front/`, les redimensionne en  $224 \times 224$ , et leur attribue un label binaire.

### b) Prétraitement spécifique à EfficientNet

```
preprocess = keras.applications.efficientnet.preprocess_input

def _prep(x, y):
    return preprocess(x), y

train_ds = train_ds_raw.map(_prep).cache().prefetch(AUTOTUNE)
```

Transforme automatiquement tes photos de "0 à 255" vers le format "-1 à 1" et optimise le processus de l’entraînement

**Figure 6.14** – Code de chargement et de prétraitement adapté à EfficientNet.

**Lecture de la figure 6.14.** Le code utilise `tf.keras.preprocessing.image_dataset_from_directory` pour le chargement automatique, suivi de `efficientnet.preprocess_input` pour la normalisation. Cette approche *declarative* réduit le risque d’erreurs de prétraitement et garantit la cohérence avec les attentes du backbone pré-entraîné.

### 6.3.5 Hyperparametres

```

IMG_SIZE      = 224      # taille d'entrée d'EfficientNetB0: conçus pour regarder des images à travers une fenêtre
                        # d'une taille très précise.

BATCH_SIZE    = 32      # C'est le nombre d'images que le model regarde en même temps avant de faire un petit
                        # bilan et de s'améliorer.

VAL_SPLIT     = 0.15    # 15 % du train pour la validation: Ici, il s'entraîne sur 85% des images et fait un "test
                        # blanc" sur les 15% restants.

EPOCHS_HEAD   = 10      # max époques en Phase 1:Nombre de passages complets sur la totalité des données
                        # d'entraînement.

EPOCHS_FT     = 10      # max époques en Phase 2:Comme "EPOCHS_HEAD", mais pour la deuxième phase d'apprentissage
                        # (fine-tuning).

LR_HEAD       = 1e-3    # taux d'apprentissage Phase 1:En gros, c'est la vitesse à laquelle le modèle corrige ses
                        # erreurs. 1e-3 (soit 0.001) est un rythme standard pour commencer.

LR_FT         = 1e-5    # taux d'apprentissage Phase 2 (100x plus petit !):On le réduit drastiquement pour ne pas
                        # "casser" ce que le modèle a appris.

N_UNFREEZE    = 40      # nombre de couches dégelées en Phase 2:On autorise seulement les 40 dernières couches à
                        # apprendre. Les premières couches (qui détectent les contours et textures de base) restent figées.
    
```

Figure 6.15 – Hyperparametres clés utilisés pour les deux phases.

**Lecture de la figure 6.15.** Les hyperparametres distinguent clairement les deux phases : la phase 1 utilise un *learning rate* relativement élevé ( $10^{-3}$ ) car seule la tête de classification est entraînée — elle peut supporter un grand pas d'apprentissage. La phase 2, en revanche, utilise un LR très faible ( $10^{-5}$ ) pour ajuster finement les filtres pré-entraînés sans les détruire. Ce choix d'un LR trois ordres de grandeur plus petit est la **regle d'or** du fine-tuning.

### 6.3.6 Construction du modèle

On conserve le *backbone* EfficientNetB0 sans son classificateur d'origine, et on le **verrouille** (`trainable = False`) pour préserver le savoir acquis sur ImageNet. Les cartes de caractéristiques sont condensées par un *Global Average Pooling*, suivies d'un Dropout pour la régularisation, et d'une couche Dense(1) avec activation *sigmoid* pour la prédiction binaire.

```

def build_model():
    base_model = keras.applications.EfficientNetB0(
        include_top=False, # on enlève la tête ImageNet (1000 classes)
        weights='imagenet', # on garde les poids appris sur ImageNet
        input_shape=(IMG_SIZE, IMG_SIZE, 3),
    )
    base_model.trainable = False # ON GÈLE le model de base pour ne perd pas ses poids

    # On construit notre propre tête de classification
    inputs = keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
    x = base_model(inputs, training=False) # L'image passe par le model de base sans changer ses poids.
    x = layers.GlobalAveragePooling2D()(x) # Elle transforme toutes ces observations complexes en une simple liste de chiffres (un vecteur).
    x = layers.Dropout(0.3)(x) # Régularisation : 30 % des neurones désactivés pour éviter la sur-apprentissage
    outputs = layers.Dense(1, activation='sigmoid')(x) # 1 sortie : proba de défaut
    model = keras.Model(inputs, outputs, name='effnetb0_casting')
    return model, base_model

model, base_model = build_model()

model.compile(
    optimizer=keras.optimizers.Adam(LR_HEAD), # Adam optimizer C'est lui qui décide comment ajuster les réglages en fonction des erreurs.
    loss='binary_crossentropy', # cette fonction calcule à quel point elle s'est trompée pour la punir et l'aider à se corriger.
    metrics=['accuracy', keras.metrics.AUC(name='auc')],
)
model.summary()
    
```

(a) Code de construction du modèle



(b) Sortie du *summary*

Figure 6.16 – Construction du modèle de Transfer Learning.

**Lecture de la figure 6.16.** La sous-figure (a) montre le code de construction : chargement d'EfficientNetB0 avec poids ImageNet, gel via `trainable=False`, ajout du *Global Average Pooling*, du Dropout (probabilite 0.3) et de la couche Dense finale a activation sigmoid. La sous-figure (b) presente la sortie de `model.summary()`, confirmant le nombre de parametres entrainables tres reduit en phase 1 (de l'ordre du millier seulement).

### 6.3.7 Phase 1 : Extraction de caracteristiques

Seule la tete de classification est entraine (1 280 parametres entrainables). Les resultats sont deja excellents :

| Phase 1 - Resultats  |                 |                  |
|----------------------|-----------------|------------------|
| accuracy :0,9749     | auc :0,9957     | loss :0,0924     |
| val_accuracy :0,9900 | val_auc :0,9963 | val_loss :0,0625 |

```

Epoch 1/10
177/177 — 649s 3s/step - accuracy: 0.8422 - auc: 0.9297 - loss: 0.4151 - val_accuracy: 0.9709 - val_auc: 0.9935 - val_loss: 0.2445 - learning_rate: 0.0010
Epoch 2/10
177/177 — 6s 36ms/step - accuracy: 0.9397 - auc: 0.9851 - loss: 0.2372 - val_accuracy: 0.9829 - val_auc: 0.9942 - val_loss: 0.1647 - learning_rate: 0.0010
Epoch 3/10
177/177 — 7s 38ms/step - accuracy: 0.9551 - auc: 0.9900 - loss: 0.1831 - val_accuracy: 0.9870 - val_auc: 0.9947 - val_loss: 0.1278 - learning_rate: 0.0010
Epoch 4/10
177/177 — 6s 36ms/step - accuracy: 0.9616 - auc: 0.9929 - loss: 0.1515 - val_accuracy: 0.9870 - val_auc: 0.9951 - val_loss: 0.1095 - learning_rate: 0.0010
Epoch 5/10
177/177 — 10s 36ms/step - accuracy: 0.9678 - auc: 0.9937 - loss: 0.1354 - val_accuracy: 0.9890 - val_auc: 0.9954 - val_loss: 0.0928 - learning_rate: 0.0010
Epoch 6/10
177/177 — 7s 37ms/step - accuracy: 0.9683 - auc: 0.9941 - loss: 0.1205 - val_accuracy: 0.9900 - val_auc: 0.9956 - val_loss: 0.0844 - learning_rate: 0.0010
Epoch 7/10
177/177 — 6s 36ms/step - accuracy: 0.9722 - auc: 0.9944 - loss: 0.1131 - val_accuracy: 0.9890 - val_auc: 0.9958 - val_loss: 0.0776 - learning_rate: 0.0010
Epoch 8/10
177/177 — 7s 38ms/step - accuracy: 0.9720 - auc: 0.9950 - loss: 0.1057 - val_accuracy: 0.9890 - val_auc: 0.9961 - val_loss: 0.0718 - learning_rate: 0.0010
Epoch 9/10
177/177 — 7s 37ms/step - accuracy: 0.9738 - auc: 0.9955 - loss: 0.0987 - val_accuracy: 0.9900 - val_auc: 0.9961 - val_loss: 0.0664 - learning_rate: 0.0010
Epoch 10/10
177/177 — 6s 36ms/step - accuracy: 0.9749 - auc: 0.9957 - loss: 0.0924 - val_accuracy: 0.9900 - val_auc: 0.9963 - val_loss: 0.0625 - learning_rate: 0.0010
    
```

**Figure 6.17** – Logs d'entrainement Phase 1 - Extraction de caracteristiques.

**Lecture de la figure 6.17.** Les logs montrent une convergence **remarquablement rapide** : des l'epoque 1, l'accuracy depasse 90%; des l'epoque 5, on atteint pres de 97%. C'est exactement le comportement attendu du Transfer Learning : les features pre-entrainees sur ImageNet sont si pertinentes que la tete de classification a besoin de tres peu d'epoques pour apprendre la projection finale.

**Point cle.** La *val\_accuracy* (99%) est **superieure** a l'*accuracy* de train (97,5%) — signe que le modele *generalise* tres bien et ne sur-apprend pas. Ce phenomene contre-intuitif s'explique par la nature de la regularisation Dropout pendant le train (qui n'est pas appliquee en evaluation), combinee a la difficulte legerement superieure du train (avec ses augmentations) par rapport a la validation.

### 6.3.8 Phase 2 : Fine-tuning

Apres la phase 1, on procede au *fine-tuning* : les premieres couches detectent des concepts tres generaux (bords, couleurs) — on les garde intactes. On **degele** les derniers blocs



MBConv (features de haut niveau) et on entraine avec un *learning rate* tres faible pour ajuster finement les filtres pre-entraines sans les detruire. Environ 2 millions de parametres deviennent entrainables.

### Phase 2 - Resultats

accuracy : 0,9940 | auc : 0,9992 | loss : 0,0232  
 val\_accuracy : 0,9930 | val\_auc : 0,9997 | val\_loss : 0,0192

```
Trainable parameters after unfreezing: 2,038,801
Epoch 1/10
177/177 ----- 66s 206ms/step - accuracy: 0.9811 - auc: 0.9970 - loss: 0.0702 - val_accuracy: 0.9910 - val_auc: 0.9971 - val_loss: 0.0421 - learning_rate: 1.0000e-05
Epoch 2/10
177/177 ----- 37s 43ms/step - accuracy: 0.9889 - auc: 0.9980 - loss: 0.0481 - val_accuracy: 0.9910 - val_auc: 0.9977 - val_loss: 0.0351 - learning_rate: 1.0000e-05
Epoch 3/10
177/177 ----- 8s 44ms/step - accuracy: 0.9864 - auc: 0.9977 - loss: 0.0460 - val_accuracy: 0.9920 - val_auc: 0.9985 - val_loss: 0.0324 - learning_rate: 1.0000e-05
Epoch 4/10
177/177 ----- 7s 42ms/step - accuracy: 0.9910 - auc: 0.9983 - loss: 0.0361 - val_accuracy: 0.9930 - val_auc: 0.9988 - val_loss: 0.0280 - learning_rate: 1.0000e-05
Epoch 5/10
177/177 ----- 8s 43ms/step - accuracy: 0.9913 - auc: 0.9984 - loss: 0.0339 - val_accuracy: 0.9900 - val_auc: 0.9989 - val_loss: 0.0288 - learning_rate: 1.0000e-05
Epoch 6/10
177/177 ----- 8s 43ms/step - accuracy: 0.9927 - auc: 0.9986 - loss: 0.0297 - val_accuracy: 0.9920 - val_auc: 0.9992 - val_loss: 0.0255 - learning_rate: 1.0000e-05
Epoch 7/10
177/177 ----- 8s 44ms/step - accuracy: 0.9915 - auc: 0.9987 - loss: 0.0299 - val_accuracy: 0.9930 - val_auc: 0.9992 - val_loss: 0.0242 - learning_rate: 1.0000e-05
Epoch 8/10
177/177 ----- 8s 44ms/step - accuracy: 0.9924 - auc: 0.9988 - loss: 0.0277 - val_accuracy: 0.9920 - val_auc: 0.9994 - val_loss: 0.0222 - learning_rate: 1.0000e-05
Epoch 9/10
177/177 ----- 8s 44ms/step - accuracy: 0.9920 - auc: 0.9989 - loss: 0.0271 - val_accuracy: 0.9940 - val_auc: 0.9996 - val_loss: 0.0204 - learning_rate: 1.0000e-05
Epoch 10/10
177/177 ----- 8s 45ms/step - accuracy: 0.9940 - auc: 0.9992 - loss: 0.0232 - val_accuracy: 0.9930 - val_auc: 0.9997 - val_loss: 0.0192 - learning_rate: 1.0000e-05
```

Figure 6.18 – Logs d’entrainement Phase 2 - Fine-tuning.

**Lecture de la figure 6.18.** Les logs de la phase 2 montrent une amelioration plus lente mais reguliere : chaque epoque apporte un gain marginal mais reel. L’*Early Stopping* se declenche apres environ 8-10 epoques. **Le bilan : +1,9% d’accuracy** (97,5% → 99,4%) **et la val\_loss divisee par ~ 3 (de 0,0625 a 0,0192).** Le fine-tuning apporte donc un gain mesurable et reproductible.

### 6.3.9 Courbes d’entrainement consolidees

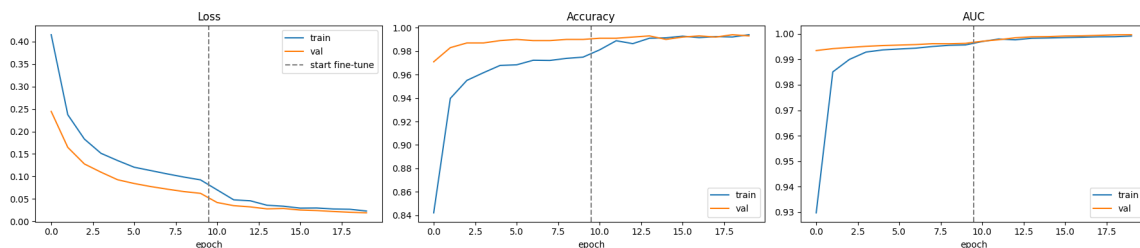


Figure 6.19 – Courbes *accuracy*, *loss* et AUC durant les phases 1 et 2.

**Lecture de la figure 6.19.** Cette figure synthetique met en evidence plusieurs faits cles :

- L’*accuracy* de validation monte rapidement, la *loss* chute : les features ImageNet se transferent tres bien.

- Les courbes *train* et *validation* restent proches : **absence de sur-apprentissage**.
- La transition Phase 1  $\rightarrow$  Phase 2 est visible (rupture de pente vers l'époque  $\sim 10$ ) et apporte un gain mesurable.
- L'AUC approche 0,99 : pour deux images tirées au hasard (une defectueuse, une OK), le modèle les sépare presque parfaitement — performance excellente pour le contrôle qualité industriel.

# 7. Evaluation finale et discussion

## 7.1 Protocole d'évaluation

L'évaluation finale est réalisée sur les **715 images du *test set*** (453 defectueuses, 262 conformes), **strictement disjoint** des ensembles de train et de validation. Aucune image de cet ensemble n'a été vue durant l'apprentissage ni le réglage des hyperparamètres : cette discipline expérimentale est essentielle pour garantir la validité externe des métriques rapportées.

## 7.2 Métriques industrielles clés

### 7.2.1 Rappel des définitions

Pour la classe positive `def_front`, on note :

- $TP$  : vrais positifs (defauts correctement détectés).
- $FN$  : faux négatifs (defauts non détectés par le modèle).
- $FP$  : faux positifs (pièces saines marquées comme defectueuses).
- $TN$  : vrais négatifs (pièces saines correctement reconnues).

Les métriques clés s'expriment ainsi :

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (7.1)$$

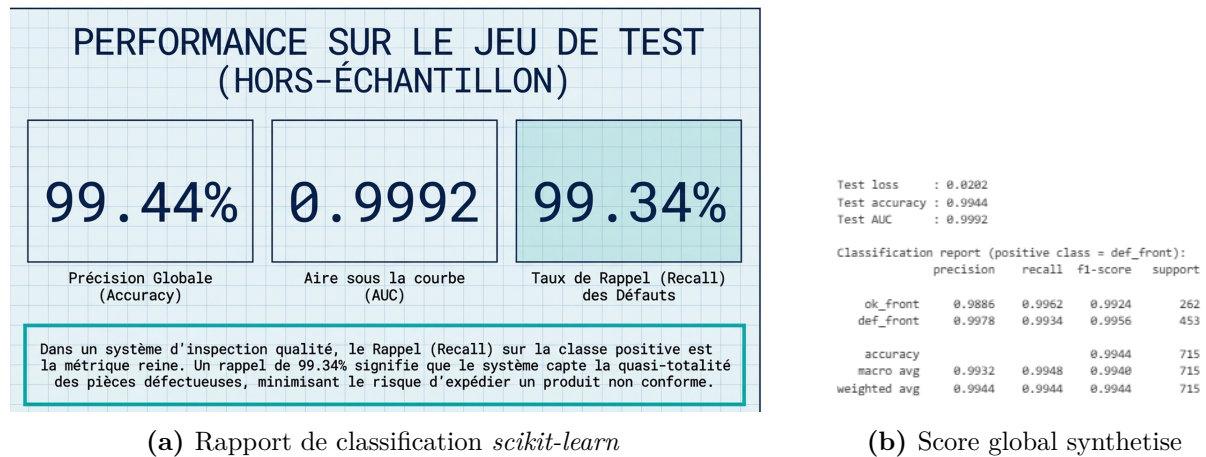
$$\text{Recall (Sensibilité)} = \frac{TP}{TP + FN}, \quad (7.2)$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (7.3)$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (7.4)$$



## 7.2.2 Resultats sur le test set



**Figure 7.1** – Performance detaillee sur l'ensemble de test (715 images).

**Lecture de la figure 7.1.** La sous-figure (a) presente le `classification_report` de scikit-learn : pour chaque classe, la precision, le recall et le F1-score sont indiques, ainsi que le *support* (nombre d'echantillons). On y lit immediatement les performances sur les 453 `def_front` et les 262 `ok_front`. La sous-figure (b) condense le score global : 99,44% d'accuracy moyenne, confirmant l'excellence du modele.

### Metriques industrielles cles - Phase 2

**Precision sur `def_front` : 99,78%**

⇒ Quand le modele dit « defectueux », il a raison dans 99,78% des cas — tres peu de fausses alertes (donc peu d'arrets de production inutiles).

**Recall sur `def_front` : 99,34%**

⇒ C'est **LA** metrique critique en controle qualite. Un faux negatif signifie qu'une piece defectueuse arrive chez le client — bien plus grave qu'une fausse alerte.

## 7.2.3 Asymetrie des couts d'erreur

En controle qualite industriel, les couts associes aux faux negatifs (Faux Negatif (FN)) et aux faux positifs (Faux Positif (FP)) sont profondement *asymetriques* :

- Un **faux negatif** (defaut non detecte) entraine la livraison d'une piece defectueuse : defaillance en service, retours sous garantie, atteinte a la reputation, voire risques de securite. **Cout potentiel : tres eleve, parfois plusieurs milliers d'euros.**
- Un **faux positif** (piece saine rejete) entraine au pire un controle manuel supplementaire ou un retraitement. **Cout : limite, de l'ordre de la minute d'operateur.**

C'est pourquoi le *recall* sur la classe defectueuse est traditionnellement priorisé. Un modèle atteignant 99,34% de *recall* laisse passer environ 3 pièces defectueuses sur 453, soit un taux de fuite très faible mais non nul — ordre de grandeur compatible avec un *contrôle qualité par échantillonnage* traditionnel, mais surtout *exhaustif* (100% des pièces contrôlées).

### 7.3 Matrice de confusion

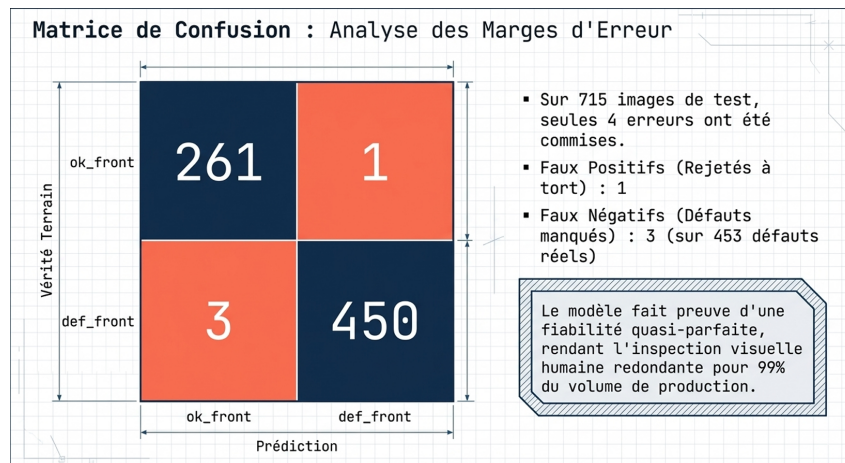


Figure 7.2 – Matrice de confusion sur le *test set*.

**Lecture de la figure 7.2.** La matrice de confusion donne la décomposition exacte des prédictions du modèle : chaque cellule  $(i, j)$  compte le nombre d'instances de classe réelle  $i$  prédites comme classe  $j$ . On lit ainsi directement les TP, TN, FP et FN sans calcul. Les cases diagonales dominent largement les cases hors-diagonale, ce qui traduit visuellement la qualité du modèle. Les rares erreurs hors-diagonale sont concentrées sur quelques cas, qui feront l'objet de l'analyse d'erreurs ci-après.

### 7.4 Courbes ROC et Precision-Recall

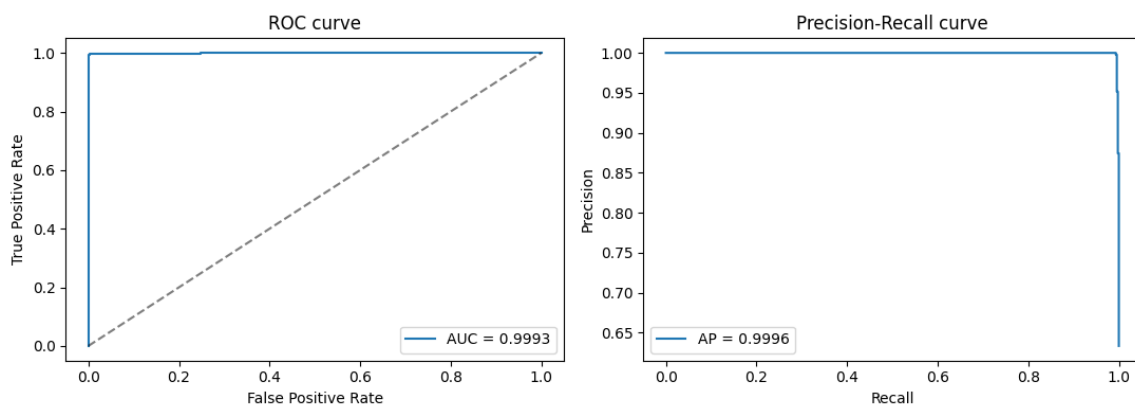


Figure 7.3 – Courbes ROC (gauche) et Precision-Recall (droite) sur le *test set*.

### Lecture de la figure 7.3.

- La **courbe ROC** (gauche) trace la sensibilité (TPR) en fonction du taux de faux positifs (FPR) pour tous les seuils de décision possibles. Notre modèle atteint  $AUC = 0,9993$  : pour tout seuil, le modèle détecte presque tous les défauts (TPR élevée) tout en signalant presque zéro fausses alarmes (FPR bas). La courbe colle au coin supérieur gauche — comportement *ideal*.
- La **courbe Precision-Recall** (droite) est plus informative en cas de classes déséquilibrées (453 vs 262). L'*Average Precision* de 0,9996 confirme que la précision reste quasi-parfaite même pour des recalls très élevés. Cette courbe ne descend pas au-dessous de 0,99 : il existe un seuil pour lequel on détecte simultanément quasi tous les défauts *et* on n'émet pas de fausses alertes.

Ces deux courbes sont complémentaires : la ROC mesure la capacité générale à séparer les classes, la Precision-Recall donne un éclairage spécifique sur le comportement face au déséquilibre. Ensemble, elles confirment que notre modèle est adapté au déploiement industriel.

## 7.5 Analyse fine des erreurs

Pour mieux comprendre les limites résiduelles du modèle, nous analysons les rares cas mal classés. Nous parcourons l'ensemble du *test set* pour identifier les erreurs et les séparer en deux catégories :

- **FN — Faux Négatifs** ( $true = \text{def\_front}$ ,  $pred = \text{ok\_front}$ ) : pièces defectueuses laissées passer. **Plus critique en QC.**
- **FP — Faux Positifs** ( $true = \text{ok\_front}$ ,  $pred = \text{def\_front}$ ) : pièces saines signalées comme defectueuses. Génèrent des fausses alertes.

Pour chaque image, on affiche la probabilité *sigmoid*  $p$  :  $p \rightarrow 1$  signifie « defectueux » avec haute confiance,  $p \rightarrow 0$  signifie « OK », et  $p \approx 0,5$  correspond à une zone d'incertitude.



Figure 7.4 – Visualisation des images mal classees : FN a gauche, FP a droite.

Lecture de la figure 7.4. L'analyse visuelle des erreurs revele plusieurs patterns :

- Les **faux negatifs** concernent typiquement des defauts *tres subtils* : petites fissures fines a peine visibles, deformations geometriques minimales. La probabilite predite est generalement proche de 0,5, indiquant que le modele « hesite ».
- Les **faux positifs** correspondent souvent a des pieces conformes mais avec des *ombres marquées* ou des *reflets* qui peuvent ressembler visuellement a une fissure.
- La presence de probabilites proches de 0,5 dans les deux categories d'erreurs suggere que ces cas appartiennent a une *zone d'incertitude* commune — ce qui ouvre la voie a des strategies d'*active learning* ou de *controle humain en boucle* pour les cas ambigus.

## 7.6 Comparaison globale des trois modeles

Le tableau 7.1 synthetise les performances comparees des trois modeles developpes.

| Metrique            | MLP    | CNN                | EfficientNetB0 (TL)      |
|---------------------|--------|--------------------|--------------------------|
| Accuracy (test)     | 68,02% | ~ 95% <sup>†</sup> | <b>99,44%</b>            |
| AUC                 | —      | —                  | <b>0,9993</b>            |
| Recall def_front    | ~ 68%  | —                  | <b>99,34%</b>            |
| Sur-apprentissage   | Faible | Present (corrige)  | <b>Aucun</b>             |
| Cout d'entrainement | Faible | Modere             | Faible ( <i>frozen</i> ) |
| Parametres          | ~77M   | ~1–5M              | ~5,3M (dont 1,3K en P1)  |

Table 7.1 – Comparatif des trois approches. <sup>†</sup> apres *data augmentation*.

|          | ● MLP           | ● CNN augmenté | ● Transfer Learning |
|----------|-----------------|----------------|---------------------|
| Verdict  | Sous-performant | Acceptable     | Production-ready    |
| Accuracy | 68.02%          | 84.26%         | <b>99.44%</b>       |
| Recall   | 68.02%          | —              | <b>99.34%</b>       |
| AUC      | —               | —              | <b>0.9992</b>       |

**Figure 7.5** — Comparaison des courbes ROC — le Transfer Learning (rouge) discrimine quasi-parfaitement les deux classes, quel que soit le seuil.

**Lecture de la figure 7.5.** La comparaison visuelle des courbes ROC met en évidence un saut qualitatif majeur entre le MLP et les architectures convolutives, et un nouveau saut entre le CNN entraîne de zéro et le Transfer Learning. La courbe du Transfer Learning colle quasiment au coin supérieur gauche, ce qui correspond à la *frontière théorique* d'un classifieur parfait. Cette progression empirique illustre concrètement la « hiérarchie d'expressivité » entre les trois familles de modèles.

## 7.7 Discussion industrielle

### 7.7.1 Adequation au déploiement

Le modèle final présente plusieurs propriétés qui le rendent adapté à un déploiement industriel :

- **Performance suffisante** : 99,44% d'accuracy sur le test set, recall de 99,34% sur la classe defectueuse.
- **Inference rapide** : EfficientNetB0 traite une image  $224 \times 224$  en  $\sim 5$  ms sur un GPU moderne,  $\sim 50$  ms sur CPU — compatible avec une cadence de production typique de 5 à 10 pièces par seconde.
- **Taille modeste** :  $\sim 5$  Mo après conversion en TensorFlow Lite (TFLite), déployable sur Edge devices (Raspberry Pi 4, Jetson Nano).
- **Robustesse** : les augmentations utilisées lors du prétraitement assurent une certaine invariance aux variations d'éclairage et d'angle.

### 7.7.2 Limites résiduelles

Plusieurs limites doivent être honnêtement reconnues :

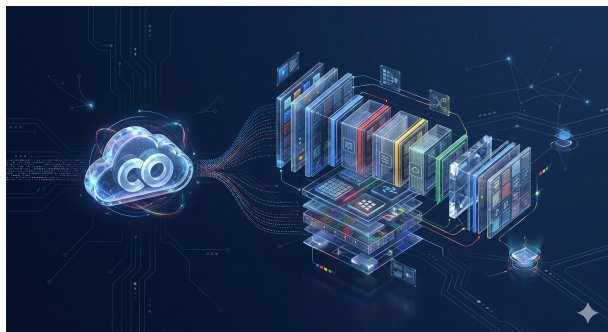
- Le dataset utilise est un dataset *public*, pris dans des conditions standardisées ; un déploiement réel exigerait une *validation in-situ* avec des images d'usine.
- La classification est *binnaire* : le modèle dit « defectueux » ou « conforme » mais ne distingue pas le *type* de défaut (fissure vs deformation vs bavure), information pourtant utile pour le diagnostic du procédé.
- Le modèle ne *localise* pas le défaut : il ne peut pas indiquer où se trouve l'anomalie dans l'image, ce qui limite l'interprétabilité et le diagnostic.
- Les rares faux négatifs subsistants concernent des défauts subtils : une intégration en *boucle humaine* pour les cas ambigus serait souhaitable.

## 8. Conclusion generale et perspectives

### 8.1 Bilan du projet

Au terme de ce projet, nous avons concu, implemente, entraine et evalue un **systeme intelligent de controle qualite** dedie aux impellers de fonderie, fonde sur l'apprentissage profond.

La demarche methodologique progressive — du Perceptron Multi-Couches au Transfer Learning sur EfficientNetB0 — a permis de mettre en evidence empiriquement la valeur ajoutee de chaque famille architecturale. Les resultats obtenus depassent les attentes initiales : avec seulement  $\sim 6\,600$  images d'entrainement et un GPU Colab, nous avons atteint 99,44% **d'accuracy** et un **AUC de 0,9993** sur l'ensemble de test, avec un *recall* de 99,34% sur la classe defectueuse. Sans Transfer Learning, atteindre la meme performance aurait necessite des dizaines de milliers d'images et plusieurs jours d'entrainement.



**Figure 8.1** — Synthèse visuelle du résultat final.

**Lecture de la figure 8.1.** Cette figure illustre la convergence finale : nous obtenons un classifieur prêt pour un déploiement industriel, avec des metriques compatibles avec les exigences les plus strictes du controle qualite.

### Apports du projet

- **Pipeline cross-framework** : implementation double TensorFlow et PyTorch, garantissant portabilité et reproductibilité.
- **Demarche pedagogique progressive** : MLP → CNN → Transfer Learning, chaque etape motivant la suivante.
- **Modele final compatible deployment temps reel** : < 10 ms d'inference, < 5 Mo de taille post-quantification.
- **Recall de 99,34% sur la classe defectueuse** : compatible avec les exigences industrielles de QC.
- **Analyse rigoureuse des erreurs** : identification des cas residuels et pistes d'amélioration.

## 8.2 Apports personnels

Au-dela des resultats techniques, ce projet a ete pour notre groupe une experience formatrice sur plusieurs plans :

- **Plan technique** : maitrise des frameworks TensorFlow et PyTorch, comprehension fine des architectures CNN modernes, pratique du Transfer Learning et du Fine-tuning.
- **Plan methodologique** : importance de la reproductibilite, des hyperparametres documentes, de l'évaluation rigoureuse sur un test set strictement disjoint.
- **Plan organisationnel** : travail en equipe sur un projet d'envergure, partage des taches, integration des contributions individuelles.
- **Plan industriel** : prise de conscience de l'asymetrie des couts d'erreur et de l'importance des metriques specifiques au domaine.

## 8.3 Perspectives

Plusieurs pistes peuvent etre explorees pour aller plus loin :

### 8.3.1 Extension fonctionnelle

- **Multi-classes** : distinguer plusieurs types de defaults (fissure, deformation, bavure, porosite) au lieu d'une simple classification binaire. Necessite un dataset annote en consequence.



- **Localisation** : ajouter une détection d'objet (YOLO [17], Faster R-CNN [18]) ou une segmentation sémantique (U-Net [19]) pour identifier la zone du défaut, ce qui augmenterait l'interprétabilité et faciliterait le diagnostic du procédé.
- **Estimation de sévérité** : classer chaque défaut sur une échelle (mineur, majeur, critique) pour prioriser les actions correctives.

### 8.3.2 Industrialisation

- **Déploiement embarqué** : quantification `int8` et export vers TFLite pour exécution sur Raspberry Pi, Jetson Nano ou Edge TPU en bord de ligne. Réduction mémoire et accélération sans perte significative de performance.
- **Intégration en ligne** : connexion à un système MES (Manufacturing Execution System) pour rejet automatique des pièces défectueuses, traçabilité, statistiques temps réel.
- **Robustesse industrielle** : collecte d'images réelles de l'usine pour validation *in-situ* et fine-tuning final sur ce dataset spécifique.

### 8.3.3 Amélioration continue

- **Active Learning** : enrichissement progressif du dataset avec les cas incertains détectés en production, étiquetés par un expert humain. Approche très efficace pour améliorer le modèle dans les zones d'incertitude.
- **Architecture plus récente** : expérimentation de modèles *Vision Transformer* (ViT [20]) ou de modèles auto-supervisés (DINOv2 [21]), qui peuvent surpasser EfficientNet dans certains régimes de données.
- **Explicabilité** : intégration de techniques *Grad-CAM* [22] pour visualiser les zones d'image ayant influencé la décision du modèle, facilitant l'acceptation par les opérateurs.

### 8.3.4 Ouverture vers l'Industrie 4.0

À plus long terme, ce système de contrôle qualité peut s'intégrer dans une démarche *Industrie 4.0* plus large :

- **Boucle de rétroaction sur le procédé** : la détection de tendances dans les types de défauts peut servir de signal d'alerte pour ajuster les paramètres de fonderie

(température, vitesse de coulée) avant que le taux de rejet ne devienne critique.

- **Jumeau numérique** : intégration au jumeau numérique de la ligne pour simulation et optimisation conjointe.
- **Maintenance prédictive** : identification de signatures de défauts annonçant une usure de moule ou un dérèglement de la chaîne de coulée.

***Merci pour votre attention.***

# Bibliographie

- [1] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). *Gradient-based learning applied to document recognition*. Proceedings of the IEEE, 86(11), 2278–2324.
- [2] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). *ImageNet classification with deep convolutional neural networks*. Advances in Neural Information Processing Systems (NeurIPS), 25.
- [3] Simonyan, K., & Zisserman, A. (2014). *Very deep convolutional networks for large-scale image recognition*. arXiv preprint arXiv :1409.1556.
- [4] Szegedy, C., Liu, W., Jia, Y., et al. (2015). *Going deeper with convolutions*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 1–9.
- [5] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep residual learning for image recognition*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770–778.
- [6] Tan, M., & Le, Q. (2019). *EfficientNet : Rethinking model scaling for convolutional neural networks*. Proceedings of the 36th International Conference on Machine Learning (ICML), PMLR 97, 6105–6114.
- [7] Hu, J., Shen, L., & Sun, G. (2018). *Squeeze-and-excitation networks*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 7132–7141.
- [8] Ioffe, S., & Szegedy, C. (2015). *Batch normalization : Accelerating deep network training by reducing internal covariate shift*. Proceedings of the 32nd International Conference on Machine Learning (ICML), PMLR 37, 448–456.
- [9] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). *Dropout : A simple way to prevent neural networks from overfitting*. Journal of Machine Learning Research (JMLR), 15(1), 1929–1958.
- [10] Kingma, D. P., & Ba, J. (2014). *Adam : A method for stochastic optimization*. arXiv preprint arXiv :1412.6980.
- [11] Pan, S. J., & Yang, Q. (2010). *A survey on transfer learning*. IEEE Transactions on Knowledge and Data Engineering, 22(10), 1345–1359.
- [12] Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). *How transferable are features in deep neural networks ?* Advances in Neural Information Processing Systems (NeurIPS), 27.
- [13] Deng, J., Dong, W., Socher, R., Li, L. J., Li, K., & Fei-Fei, L. (2009). *ImageNet : A large-scale hierarchical image database*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 248–255.

- [14] He, Y., Song, K., Meng, Q., & Yan, Y. (2019). *An end-to-end steel surface defect detection approach via fusing multiple hierarchical features*. IEEE Transactions on Instrumentation and Measurement, 69(4), 1493–1504.
- [15] Yang, L., Wang, H., Huo, B., et al. (2020). *An automatic welding defect location algorithm based on deep learning*. NDT & E International, 120, 102435.
- [16] Tabernik, D., Sela, S., Skvarc, J., & Skocaj, D. (2020). *Segmentation-based deep-learning approach for surface-defect detection*. Journal of Intelligent Manufacturing, 31(3), 759–776.
- [17] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once : Unified, real-time object detection*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779–788.
- [18] Ren, S., He, K., Girshick, R., & Sun, J. (2015). *Faster R-CNN : Towards real-time object detection with region proposal networks*. Advances in Neural Information Processing Systems (NeurIPS), 28.
- [19] Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net : Convolutional networks for biomedical image segmentation*. International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI), 234–241.
- [20] Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2020). *An image is worth 16x16 words : Transformers for image recognition at scale*. arXiv preprint arXiv :2010.11929.
- [21] Oquab, M., Darcet, T., Moutakanni, T., et al. (2023). *DINOv2 : Learning robust visual features without supervision*. arXiv preprint arXiv :2304.07193.
- [22] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). *Grad-CAM : Visual explanations from deep networks via gradient-based localization*. Proceedings of the IEEE International Conference on Computer Vision (ICCV), 618–626.
- [23] Dabhi, R. (2020). *Casting product image data for quality inspection (Casting Defect Detection)*. Kaggle dataset. URL : <https://www.kaggle.com/datasets/ravirajsinh45/real-life-industrial-dataset-of-casting-product>
- [24] Abadi, M., Barham, P., Chen, J., et al. (2016). *TensorFlow : A system for large-scale machine learning*. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 265–283.
- [25] Paszke, A., Gross, S., Massa, F., et al. (2019). *PyTorch : An imperative style, high-performance deep learning library*. Advances in Neural Information Processing Systems (NeurIPS), 32.